



Universitat
de les Illes Balears

TRABAJO DE FIN DE GRADO

TRABAJO DE FIN DE GRADO

David Vázquez Rivas

Grado en Ingeniería Informática

Escola Politècnica Superior

2025-2026

TRABAJO DE FIN DE GRADO

David Vázquez Rivas

Trabajo de Fin de Grado

Escola Politècnica Superior

Universitat de les Illes Balears

2025-2026

Palabras clave del trabajo: Gestión de Proyectos, Inteligencia Artificial

Tutor: Raimon Jaume Massanet Vila

ÍNDICE GENERAL

Índice general	i
1 Acrónimos	1
2 Introducción	3
2.1 Contexto	3
2.2 Motivación	3
2.3 Objetivos	4
2.3.1 Objetivo General	4
2.3.2 Objetivos Específicos	4
3 Análisis	5
3.1 Interesados	5
3.1.1 Usuarios	5
3.1.2 Stakeholders?	5
3.2 Especificación funcional	5
3.3 Especificación no funcional	9
3.4 Modelado del Dominio	11
3.4.1 Lenguaje ubicuo	11
3.4.2 Modelo de Datos	13
4 Diseño	23
4.1 Arquitectura de la aplicación	23
4.1.1 Diagrama de contexto (Nivel 1)	23
4.1.2 Diagrama de contenedores (Nivel 2)	23
4.1.3 Diagrama de componentes (Nivel 3)	25
4.2 Patrón de despliegue	29
4.2.1 Orquestación con Docker Compose	29
4.2.2 Despliegue del cliente web	30
4.3 Diagramas de secuencia	30
4.3.1 Sincronización de datos entre módulos	30
4.3.2 Ejecución de tareas de los agentes de Inteligencia Artificial (IA)	31
4.3.3 Flujo de autenticación	33
4.4 Patrones de diseño	35
4.4.1 Patrones creacionales	35
4.4.2 Patrones estructurales	39
4.4.3 Patrones de comportamiento	43

4.5	Diseño de la interfaz de usuario	50
4.5.1	Decisiones de diseño e implementación	50
4.5.2	Visualización de datos y complejidad	50
4.5.3	Flujo de comunicación con el usuario	50
5	Planificación	53
5.1	Gestión del ciclo de vida	53
5.1.1	Organización del proyecto y Sprints	53
5.1.2	Diagrama de Gantt	54
5.1.3	Flujo de trabajo y seguimiento de tareas	55
5.2	Aseguramiento de la calidad y pruebas	55
5.3	Definición del <i>Backlog</i>	55
5.3.1	Estructuración y jerarquía de elementos	56
5.3.2	Criterios de priorización	56
5.3.3	Estimación de esfuerzo	56
5.4	<i>Definition of Ready</i> y <i>Definition of Done</i>	56
5.4.1	<i>Definition of Ready</i>	57
5.4.2	<i>Definition of Done</i>	57
5.5	Camino crítico y análisis de dependencias	58
5.6	Políticas de versionado y gestión del código fuente	59
5.6.1	Modelo teórico y políticas de protección	59
5.6.2	Adaptación pragmática: <i>Trunk-Based Development</i>	60
A	Anexos	61
	Bibliografía	63

CAPÍTULO 1

ACRÓNIMOS

IA Inteligencia Artificial

LLM Large Language Model

UX Experiencia de Usuario

MVP Producto Mínimo Viable

PM Project Manager

CI Integración Continua

DDD Domain-Driven Design

MER Modelo Entidad-Relación

CQRS Command Query Responsibility Segregation

API Interfaz de Programación de Aplicaciones

SPA Single Page Application

1. ACRÓNIMOS

MVC Modelo-Vista-Controlador

HTTP Protocolo de Transferencia de Hipertexto

VPS Servidor Privado Virtual

PaaS Plataforma como Servicio

CDN Red de Distribución de Contenidos

CI/CD Integración y Despliegue Continuos

UML Lenguaje de Modelado Unificado

LLM Modelo de Lenguaje Grande

XSS Cross-Site Scripting

CSRF Cross-Site Request Forgery

SSO Single Sign-On

JWT JSON Web Token

UI Interfaz de Usuario

UX Experiencia de Usuario

TDD Desarrollo Guiado por Pruebas

MCP Model Context Protocol

CPM Critical Path Method

INTRODUCCIÓN

2.1 Contexto

TODO

2.2 Motivación

Aunque las herramientas actuales de gestión han integrado con éxito visualizaciones clásicas, estas en su mayoría perpetúan paradigmas analógicos. Esta herencia visual desaprovecha las capacidades de las interfaces modernas para sintetizar la complejidad, lo que genera dos barreras principales en la industria actual:

- **Alta carga cognitiva y fatiga visual:** Para comprender el estado real de un proyecto, los gestores y desarrolladores deben procesar manualmente una gran cantidad de texto y datos abstractos dispersos. La falta de representaciones visuales intuitivas dificulta la detección rápida de bloqueos, dependencias o riesgos, obligando al usuario a realizar un esfuerzo mental considerable para construir un modelo mental del proyecto.
- **Burocracia frente a trabajo de valor:** Gran parte del tiempo de gestión se invierte en tareas mecánicas y repetitivas: solicitar actualizaciones de estado, requerir la cumplimentación de campos y sintetizar información fragmentada. Esta microgestión interrumpe los estados de concentración de los perfiles técnicos y desvía a los responsables de la toma de decisiones estratégicas.

2.3 Objetivos

2.3.1 Objetivo General

Investigar, diseñar y desarrollar un nuevo paradigma de plataforma de gestión de proyectos que reemplace las vistas tabulares convencionales por metáforas visuales inmersivas, capaces de representar la complejidad estructural del trabajo. Asimismo, se pretende concebir e integrar un sistema inteligente capaz de actuar de forma autónoma para asistir en la coordinación asíncrona y el seguimiento de los equipos.

2.3.2 Objetivos Específicos

Para alcanzar el objetivo general, se plantean los siguientes objetivos específicos:

- **A. Investigación y análisis:** Analizar las limitaciones de usabilidad y Experiencia de Usuario (UX) en las herramientas de gestión convencionales y estudiar su impacto en la carga cognitiva. Investigar patrones de diseño de interfaces en sectores ajenos a la gestión empresarial que permitan representar jerarquías y dependencias de forma intuitiva.
- **B. Diseño conceptual y visual:** Definir una propuesta de interfaz gráfica y experiencia de usuario que permita visualizar la topología de un proyecto como un sistema interconectado. El objetivo es priorizar la claridad visual frente a la lectura extensiva de texto, fomentando una interacción más natural y adaptada a entornos profesionales.
- **C. Sistema de agentes:** Diseñar un sistema basado en IA capaz de actuar como intermediario ágil entre el usuario y la plataforma. El propósito es establecer flujos de trabajo que permitan delegar tareas repetitivas de coordinación, comunicación asíncrona y seguimiento de estados.
- **D. Arquitectura de software:** Diseñar e implementar una base técnica robusta que garantice la escalabilidad, el alto rendimiento y la mantenibilidad del código. Se persigue estructurar el sistema aplicando principios de diseño de software avanzados que aseguren un alto grado de desacoplamiento entre los dominios de la aplicación, facilitando su evolución y mantenimiento futuros.
- **E. Prototipado y validación técnica:** Desarrollar un Producto Mínimo Viable (MVP) que integre los nuevos paradigmas de visualización y el sistema multi-agente. Validar la viabilidad de la arquitectura propuesta y evaluar si la solución aporta un valor funcional tangible frente a los enfoques de gestión de proyectos tradicionales.

ANÁLISIS

3.1 Interesados

3.1.1 Usuarios

Se han identificado dos tipos de usuarios principales para la aplicación: los gestores de proyectos y los miembros del equipo de proyecto. Ambos tipos de usuarios son agrupados bajo el término genérico de "usuario", ya que ambos interactúan con la aplicación, aunque con diferentes objetivos y necesidades.

Gestores: Son los usuarios encargados de la dirección y supervisión del proyecto. Tienen visión global del proyecto. A nivel de rol, puede ser un Project Manager (PM) tradicional, pero también puede ser un rol de liderazgo técnico (Tech Lead, Product Owner, Arquitecto de Software, etc.) que asuma responsabilidades de gestión de proyecto.

Miembros del equipo: Son los usuarios encargados de la ejecución de las tareas del proyecto. Su interés principal en la aplicación es que el overhead de gestión sea mínimo, y que les permita centrarse en su trabajo. A nivel de rol, pueden ser desarrolladores, diseñadores, analistas, incluso agentes de IA que colaboren en el proyecto.

3.1.2 Stakeholders?

3.2 Especificación funcional

La especificación de los requisitos funcionales se ha realizado con la técnica de historias de usuario, que permite describir las funcionalidades de la aplicación desde la perspectiva del usuario final. Cada historia de usuario se ha formulado siguiendo el formato:

“Como [*tipo de usuario*], quiero [*funcionalidad*] para [*beneficio*].”

3. ANÁLISIS

Además, cada historia de usuario se ha priorizado utilizando la técnica MoSCoW, que clasifica las funcionalidades en *Must Have* (imprescindibles), *Should Have* (deseables), *Could Have* (interesantes) y *Won't Have* (no necesarias).

También se han organizado las historias de usuario en *épicas*, que agrupan funcionalidades relacionadas entre sí.

Cuadro 3.1: Historias de usuario

ID	Historia	Prioridad
Épica 1: Identidad y acceso		
HU-01	Como usuario, quiero registrarme mediante OAuth (Google, Microsoft, Facebook, X) para acceder a mi espacio de trabajo de forma segura sin gestionar nuevas contraseñas.	<i>Must Have</i>
Épica 2: Organización		
HU-02	Como gestor, quiero crear una organización que agrupe proyectos y usuarios para centralizar la gestión de mi empresa o grupo de trabajo en un solo lugar.	<i>Should Have</i>
HU-03	Como gestor, quiero definir roles y permisos de la organización para permitir a ciertos usuarios diferentes acciones sobre los proyectos de la organización.	<i>Should Have</i>
Épica 3: Configuración de proyectos		
HU-04	Como gestor, quiero crear proyectos dentro de una organización o de forma independiente para organizar el trabajo en diferentes iniciativas o clientes.	<i>Must Have</i>
HU-05	Como gestor, quiero poder configurar nombre, abreviación y privacidad de los proyectos para gestionar el acceso y la visibilidad de la información del proyecto.	<i>Must Have</i>
HU-06	Como gestor, quiero definir permisos y roles a nivel de proyecto para controlar el acceso a la información y funcionalidades del proyecto.	<i>Must Have</i>
HU-07	Como gestor, quiero invitar a usuarios a un proyecto para colaborar con mi equipo en la gestión del proyecto.	<i>Must Have</i>
HU-08	Como gestor, quiero clonar un proyecto existente para reutilizar la estructura y configuración de un proyecto anterior como plantilla en un nuevo proyecto.	<i>Could Have</i>
HU-09	Como gestor, quiero añadir campos personalizados a los elementos de trabajo del proyecto para adaptar la gestión del proyecto a las necesidades específicas de mi equipo o proyecto.	<i>Should Have</i>
Épica 4: Elementos de Trabajo		
Continúa en la siguiente página		

Tabla 3.1 – continuación		
ID	Historia	Prioridad
HU-10	Como usuario, quiero crear y gestionar elementos de trabajo para organizar y seguir el progreso de las tareas dentro de un proyecto.	<i>Must Have</i>
HU-11	Como usuario, quiero que los elementos de trabajo tengan un set de campos predefinidos (título, descripción, estado, responsable, etiquetas, etc.) para facilitar la organización y seguimiento de las tareas dentro de un proyecto.	<i>Must Have</i>
HU-12	Como usuario, quiero poder dividir el trabajo en elemento de trabajo más pequeños (subtareas) para gestionar tareas complejas dividiéndolas en partes más manejables y asignar diferentes responsables a cada parte.	<i>Must Have</i>
HU-13	Como miembro del equipo, quiero poder añadir comentarios a los elementos de trabajo y ver un historial de cambios para facilitar la comunicación y colaboración entre los miembros del equipo, y mantener un registro de las decisiones relacionadas con cada tarea.	<i>Should Have</i>
Épica 5: Notificaciones		
HU-14	Como usuario, quiero recibir notificaciones sobre eventos que me afecten (asignación de tareas, cambios en tareas asignadas, menciones, etc.) para mantenerme informado sobre el progreso del proyecto y las tareas que me afectan.	<i>Should Have</i>
HU-15	Como usuario, quiero recibir notificaciones cuando se hace una solicitud de aprobación de una acción que requiera mi aprobación para poder desbloquear el trabajo de otros miembros del equipo.	<i>Should Have</i>
Épica 6: Chats y agentes		
HU-16	Como usuario, quiero poder tener chats, tanto con otros usuarios como con agentes de IA para facilitar la comunicación y colaboración entre los miembros del equipo.	<i>Should Have</i>
HU-17	Como usuario, quiero poder crear agentes de IA que me ayuden en la gestión del proyecto y tengan un rol específico para automatizar tareas repetitivas, obtener resúmenes de información relevante, o recibir sugerencias para la gestión del proyecto.	<i>Should Have</i>
HU-18	Como gestor, quiero que los agentes de IA puedan realizar acciones dentro de la aplicación (crear elementos de trabajo, asignar tareas, enviar notificaciones, etc.) para automatizar tareas y procesos dentro de la gestión del proyecto.	<i>Should Have</i>
Continúa en la siguiente página		

3. ANÁLISIS

Tabla 3.1 – continuación		
ID	Historia	Prioridad
HU-19	Como gestor, quiero tener un agente de IA predefinido <i>Daily Standup</i> para recibir un resumen diario del estado del proyecto, incluyendo tareas pendientes, bloqueos, y progreso general.	<i>Could Have</i>
HU-20	Como gestor, quiero tener un agente de IA predefinido <i>Desglose de Trabajo</i> para poder introducir una tarea compleja y recibir una propuesta de desglose en subtareas más manejables.	<i>Could Have</i>
Épica 7: Visualización en Árbol		
Tras investigar diferentes opciones para mostrar el estado del proyecto de forma visual, se ha decidido implementar una vista en árbol que permita visualizar la estructura del proyecto y el progreso de cada elemento.		
HU-21	Como usuario, quiero visualizar el proyecto como un grafo donde los elementos de trabajo son nodos y las dependencias son conexiones para entender la estructura jerárquica y el orden de ejecución sin entrar al detalle de cada elemento de trabajo.	<i>Must Have</i>
HU-22	Como usuario, quiero identificar el tipo de trabajo por la forma del nodo y su esfuerzo estimado por el tamaño para distinguir rápidamente <i>features</i> complejas de tareas pequeñas sin leer.	<i>Must Have</i>
HU-23	Como usuario, quiero identificar el avance de cada elemento de trabajo por el relleno del nodo para conocer rápidamente el progreso.	<i>Must Have</i>
HU-24	Como usuario, quiero ver visualmente cuanto esfuerzo se ha dedicado a un elemento de trabajo respecto a lo planificado para detectar rápidamente desviaciones y poder actuar en consecuencia.	<i>Must Have</i>
HU-25	Como usuario, quiero ver en que estado se encuentra un elemento de trabajo mediante su color para coordinarme con el equipo y saber rápidamente si un elemento de trabajo está bloqueado, en progreso, o completado.	<i>Must Have</i>
HU-26	Como usuario, quiero ver quién tiene asignado un elemento de trabajo para coordinarme con el equipo.	<i>Must Have</i>
HU-27	Como usuario, quiero poder minimizar partes del árbol de trabajo para enfocarme en las partes del proyecto que me interesen sin perder la visión global.	<i>Must Have</i>
Épica 8: Perfil		
Continúa en la siguiente página		

Tabla 3.1 – continuación		
ID	Historia	Prioridad
HU-28	Como usuario, quiero tener un perfil personal indicando mi rol y zona horaria para que otros usuarios puedan conocer mi rol en los proyectos y para que las notificaciones se ajusten a mi zona horaria.	<i>Could Have</i>
HU-29	Como usuario, quiero acumular experiencia en diferentes áreas para que gestores y agentes de IA puedan recomendarme tareas o proyectos en función de mi experiencia y habilidades.	<i>Could Have</i>
HU-30	Como usuario, quiero que mi perfil contenga un indicador de calidad técnica para que los gestores y agentes de IA puedan tener en cuenta no solo mi experiencia, sino también la calidad de mis contribuciones.	<i>Could Have</i>
HU-31	Como usuario, quiero que mi perfil contenga un indicador de la velocidad a la que suelo completar tareas para que los gestores y agentes de IA puedan asignarme tareas teniendo en cuenta no solo mi experiencia y calidad, sino también mi velocidad de trabajo.	<i>Could Have</i>
HU-32	Como usuario, quiero que mi perfil contenga un indicador de mi sesgo de optimismo/pesimismo en la estimación de tareas para que los gestores y agentes de IA puedan ajustar las estimaciones de las tareas que me asignen teniendo en cuenta mi sesgo de optimismo/pesimismo.	<i>Could Have</i>

3.3 Especificación no funcional

Así mismo, se han identificado los requisitos no funcionales que rigen las restricciones, cualidades y atributos de calidad globales del sistema. Estos requisitos son críticos para garantizar el correcto funcionamiento, la robustez y la viabilidad práctica de la aplicación, abarcando dimensiones esenciales como la seguridad, el rendimiento, la escalabilidad, la disponibilidad, la mantenibilidad y la usabilidad.

Seguridad:

- **Autenticación estandarizada:** El control de acceso a la plataforma debe gestionarse mediante un sistema de autenticación seguro basado en los estándares industriales OAuth 2.0[1].
- **Control de acceso granular:** La protección de los datos e información sensible de las organizaciones y proyectos debe garantizarse mediante un modelo de control de acceso basado en roles (RBAC), aplicando restricciones jerárquicas tanto a nivel global de la organización como a nivel específico de cada proyecto.
- **Gestión de sesiones:** Las sesiones de los usuarios deben expirar automáticamente tras un periodo preconfigurado de inactividad y ofrecer la capacidad de revocar

de forma remota e instantánea cualquier sesión activa en caso de compromiso de la cuenta.

Rendimiento:

- **Tiempo de respuesta de la API:** El tiempo medio de respuesta del servidor para operaciones síncronas de lectura y escritura de datos debe mantenerse por debajo de los 300 ms para el 95 % de las solicitudes (percentil 95) bajo condiciones normales de carga.
- **Procesamiento asíncrono de IA:** Las tareas con alta carga computacional, tales como la generación de resúmenes por parte de agentes de IA o el desglose automatizado de elementos de trabajo complejos, deben procesarse de manera asíncrona, notificando al usuario el inicio del proceso.
- **Fluidez de la interfaz (Vista en árbol):** El motor de renderizado en el cliente debe asegurar una tasa de refresco fluida y cercana a los 60 fotogramas por segundo (fps) durante la interacción visual y la manipulación del grafo de tareas, minimizando el impacto del re-renderizado en proyectos complejos de hasta 500 elementos concurrentes.

Escalabilidad:

- **Escalado horizontal:** La arquitectura del sistema debe permitir el escalado horizontal independiente de sus componentes (servicios de aplicación, pasarelas de agentes de IA y bases de datos), facilitando el aislamiento de cargas de trabajo intensivas sin degradar el núcleo de la aplicación.
- **Concurrencia del sistema:** El sistema debe ser capaz de manejar de forma eficiente hasta 1000 usuarios concurrentes activos y soportar picos de carga de hasta 100 operaciones por segundo sin experimentar una degradación perceptible en los tiempos de respuesta.

Disponibilidad:

- **Tiempo de actividad (Uptime):** La aplicación debe garantizar una disponibilidad continua del 99.9 % del tiempo, excluyendo exclusivamente las ventanas de tiempo previamente notificadas para tareas de mantenimiento programado.
- **Tolerancia a fallos:** El sistema debe implementar mecanismos de recuperación automática y aislamiento de fallos, tales como políticas de reintentos, para mitigar caídas de servicios externos o de las APIs de proveedores de IA.

Mantenibilidad:

- **Arquitectura desacoplada:** El diseño del código fuente debe seguir un patrón arquitectónico modular, desacoplado y de alta cohesión (como la arquitectura hexagonal o limpia[2]), separando estrictamente la lógica del dominio de los detalles de infraestructura para facilitar su comprensión, mantenimiento y extensión.

- **Cobertura de pruebas:** Las capas de la aplicación que albergan las reglas de negocio y los casos de uso centrales deben contar con una cobertura de pruebas unitarias automatizadas de al menos un 80 %.
- **Inyección de configuración:** Todos los parámetros operativos, variables de entorno, credenciales de servicios externos e hiperparámetros de los modelos de IA deben estar completamente externalizados del código fuente, permitiendo modificaciones en caliente sin necesidad de recompilar o redesplegar la aplicación.
- **Análisis estático obligatorio:** El flujo de integración continua (Integración Continua (CI)) debe requerir la superación obligatoria de un análisis estático de código para garantizar los estándares de calidad del software, el control de la complejidad ciclomática y la ausencia de deuda técnica crítica antes de cada despliegue.

Usabilidad:

- **Heurísticas de diseño:** La interfaz de usuario deberá cumplir con los criterios de usabilidad validados mediante una evaluación heurística basada en los 10 principios de Nielsen y Molich [3].
- **Accesibilidad web:** La aplicación debe observar las directrices esenciales de accesibilidad web, asegurando relaciones de contraste cromático adecuadas en los nodos del árbol de tareas y el soporte para la navegación y operación mediante teclado en los formularios principales [4].

3.4 Modelado del Dominio

El análisis del núcleo de Leaftask se ha abordado desde la perspectiva del Domain-Driven Design (DDD) [5], priorizando la comprensión de las reglas de negocio. Dado que la gestión de proyectos asistida por inteligencia artificial presenta una alta complejidad conceptual, el dominio principal se ha descompuesto en múltiples *Bounded Contexts* [6].

En esta fase de análisis, cada *Bounded Context* [6] establece un límite estricto donde un subconjunto del modelo de dominio tiene total validez y autonomía. Esta separación permite aislar y entender áreas de responsabilidad claras (como la gestión de identidades, la estructura de los elementos de trabajo o la coordinación de agentes automatizados), garantizando que el modelo refleje fielmente la realidad del negocio antes de tomar decisiones.

A continuación, se detallan los pilares de este análisis, comenzando por el vocabulario compartido que vertebra el sistema, hasta llegar a la representación de los datos de cada contexto.

3.4.1 Lenguaje ubicuo

Siguiendo los principios del DDD[5], se ha definido un lenguaje ubicuo que unifica la terminología utilizada por los desarrolladores y los usuarios, facilitando la comunicación y el entendimiento mutuo entre ambos grupos. Este lenguaje se ha construido a

3. ANÁLISIS

partir de los términos y conceptos más relevantes para la gestión de proyectos, y se ha utilizado de forma consistente en toda la documentación, los diagramas y el código fuente de la aplicación.

Organización: Entidad principal que agrupa a múltiples usuarios y proyectos bajo un mismo espacio de trabajo, centralizando la gestión de permisos y roles a nivel corporativo o de grupo.

Proyecto: Iniciativa de trabajo acotada que contiene un conjunto estructurado de elementos de trabajo orientados a un objetivo común.

Miembro del Equipo: Entidad asignada a un proyecto para colaborar en su ejecución. Un miembro del equipo puede ser tanto un usuario humano (desarrollador, diseñador) como un Agente de IA.

Gestor: Rol encargado de la configuración, supervisión y administración de proyectos y organizaciones.

Agente de IA: Miembro del equipo automatizado que colabora en el proyecto. Actúa bajo un rol específico y tiene capacidad para interactuar con el sistema de manera análoga a un usuario humano.

Elemento de Trabajo (*Work Item*): Unidad básica de gestión dentro de un proyecto. Representa una tarea, funcionalidad o incidencia, y encapsula información como estado, responsable, esfuerzo estimado y dependencias.

Subtarea: Elemento de trabajo que se deriva o desglosa de un elemento de trabajo padre más complejo, estableciendo una relación jerárquica.

Árbol de Trabajo: Representación visual, jerárquica e interactiva del conjunto de elementos de trabajo de un proyecto en forma de grafo.

Nodo: Representación gráfica de un único Elemento de Trabajo dentro del Árbol de Trabajo. Sus atributos visuales (forma, tamaño, color y relleno) comunican instantáneamente el tipo de trabajo, el esfuerzo estimado, el estado y el avance.

Estado: Situación operativa actual de un elemento de trabajo en su ciclo de vida (pendiente, en progreso, bloqueado, completado).

Rol: Conjunto de responsabilidades y nivel de autoridad asignado a un Miembro del Equipo dentro de una Organización o Proyecto. Define qué acciones puede realizar en el sistema.

Permiso: Autorización explícita y granular para ejecutar una acción específica o acceder a un recurso concreto (ej. crear proyecto, invitar usuario, modificar elemento de trabajo). Los permisos se agrupan conformando un Rol.

Campo Personalizado: Atributo dinámico definido para extender la información predefinida de los Elementos de Trabajo, adaptándolos a las necesidades específicas de un proyecto.

Aprobación: Petición formal generada dentro del ciclo de vida de una acción que requiere la validación explícita de un usuario con los permisos adecuados para desbloquear el progreso.

3.4.2 Modelo de Datos

Siguiendo el enfoque de DDD[5], se ha diseñado un modelo conceptual de datos que refleja fielmente las entidades y relaciones del negocio. Si bien el dominio se concibe bajo un paradigma orientado a objetos, se ha optado por emplear el Modelo Entidad-Relación (MER)[7] para representar la estructura de la información que deberá ser persistente.

Dada la amplitud de la información que maneja el sistema, el modelo de datos no se presenta mediante un diagrama global. En su lugar, se ha segmentado haciendo corresponder los diagramas MER con los *Bounded Contexts* previamente identificados. Esta agrupación lógica agiliza la comprensión de los datos y asegura que cada módulo analizado mantenga una alta cohesión en torno a su contexto de negocio específico.

Modelo de Usuarios e Identidad

Este módulo es responsable de gestionar la información relacionada con los usuarios, su autenticación y sus estadísticas. Incluye las entidades:

- ***refresh_tokens***: Almacena los tokens de refresco generados para los usuarios, es importante persistirlos para poder invalidarlos en caso de compromiso de la cuenta.
- ***users***: Almacena la información básica de los usuarios, como su nombre, correo electrónico, rol y zona horaria.
- ***profile_images***: Almacena información sobre las imágenes de perfil de los usuarios como su URL y metadatos asociados.
- ***skill_types***: Almacena los tipos de habilidades que puede tener un usuario (lenguajes, metodologías, herramientas, etc.)
- ***skills***: Almacena las habilidades disponibles en el sistema, asociadas a un tipo de habilidad.
- ***user_skill_metrics***: Tabla intermedia que asocia a cada usuario con sus habilidades y contiene métricas como experiencia, calidad, velocidad y sesgo de predicción.

Módulo de Organizaciones

Este módulo gestiona la información relacionada con las organizaciones, sus miembros y roles. Incluye las entidades:

- ***organizations***: Almacena la información básica de las organizaciones, como su nombre, descripción y website.

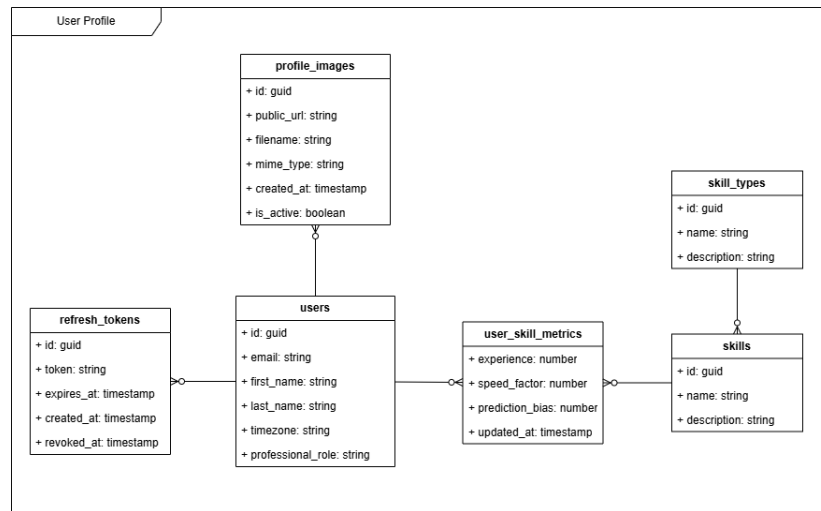


Figura 3.1: MER del módulo de Usuarios e Identidad

- **organization_invitations:** Almacena las invitaciones enviadas a usuarios para unirse a una organización, incluyendo el estado de la invitación. Los miembros activos de la organización, son los que tienen una invitación en estado aceptada.
- **organization_roles:** Almacena los roles definidos a nivel de organización, como Administrador, Gestor, Miembro, etc.
- **organization_permissions:** Almacena los permisos de organización disponibles en el sistema, como Crear Proyecto, Invitar Usuario, etc.
- **organization_role_permissions:** Tabla intermedia que asocia los roles de organización con sus permisos correspondientes y su nivel sobre estos permisos.
- **user_readmodels:** Almacena información de lectura específica de los usuarios dentro del contexto de una organización. Sigue el patrón Command Query Responsibility Segregation (CQRS)[8] que se detallará más adelante en el diseño de la arquitectura.

Módulo de Proyectos

Este módulo gestiona la información relacionada con los proyectos, sus miembros, roles, elementos de trabajo y otras configuraciones específicas de cada proyecto. Incluye las entidades:

- **projects:** Almacena la información básica de los proyectos, como su nombre, abreviación, privacidad, etc.
- **organization_readmodels:** Almacena información de lectura de las organizaciones para conocer la relación entre proyectos y organizaciones.
- **user_readmodels:** Almacena información de lectura de los usuarios para conocer su relación con los proyectos. Aquí encontramos un polimorfismo ya que un

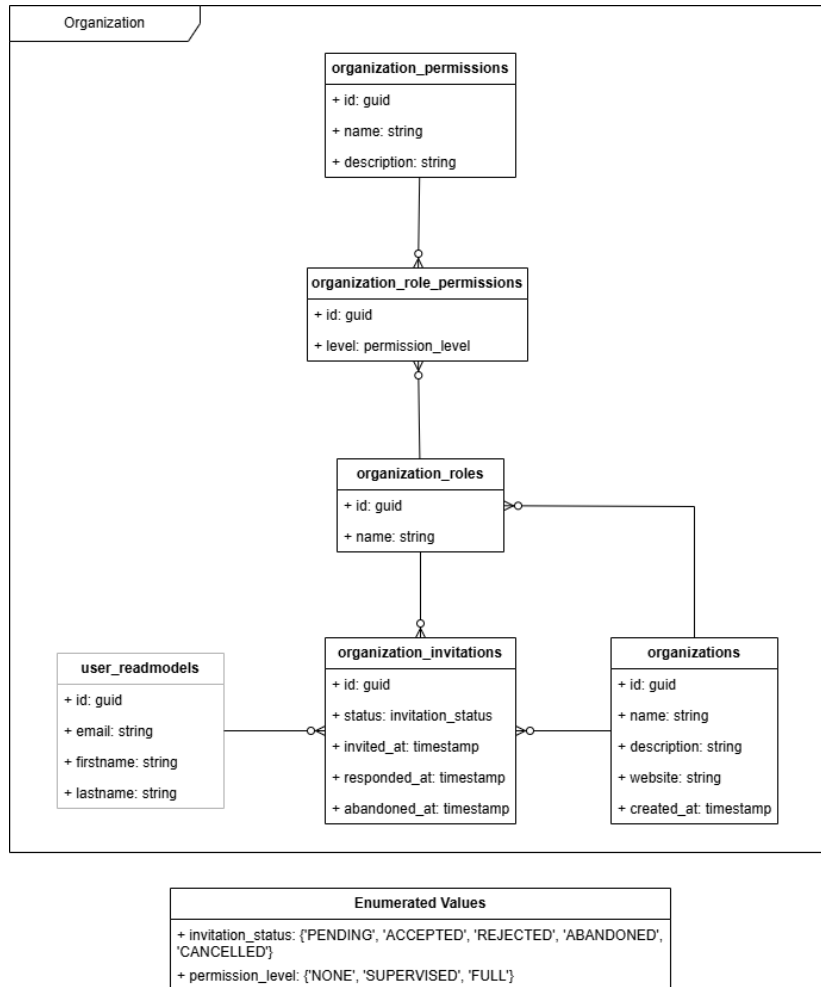


Figura 3.2: MER del módulo de Organizaciones

proyecto puede pertenecer a una organización o a un usuario de forma independiente.

- **agent_readmodels**: Almacena información de lectura de los agentes de IA para conocer los pertenecientes a un proyecto y su rol dentro de este.
- **project_invitations**: Almacena las invitaciones para unirse a un proyecto, los miembros activos de un proyecto son los que tienen una invitación en estado aceptada. Aquí también encontramos un polimorfismo ya que se pueden invitar tanto usuarios humanos como agentes de IA.
- **project_roles**: Almacena los roles definidos a nivel de proyecto, como Gestor, Desarrollador, etc.
- **project_permissions_groups**: Almacena los grupos de permisos de proyecto disponibles en el sistema, únicamente se han definido para diferenciar visualmente los permisos que afectan a la gestión del proyecto de los que afectan a la gestión de los elementos de trabajo.

- ***project_permissions***: Almacena los permisos de proyecto disponibles en el sistema, como Configurar Proyecto, Crear Elemento de Trabajo, etc.
- ***project_role_permissions***: Tabla intermedia que asocia los roles de proyecto con sus permisos correspondientes y su nivel sobre estos permisos.
- ***field_types***: Almacena los tipos de campos personalizados disponibles en el sistema, como texto, número, fecha, etc.
- ***fields***: Almacena los campos personalizados de los proyectos, incluyendo su tipo y nombre.
- ***project_fields***: Tabla intermedia que asocia los campos personalizados con los proyectos a los que pertenecen.
- ***options***: Almacena las opciones de los campos personalizados de tipo selección, incluyendo su valor y su contenido.
- ***work_item_type_readmodels***: Almacena información de lectura de los tipos de elementos de trabajo, para poder relacionarlos con los campos personalizados.
- ***field_applies***: Tabla intermedia que asocia los campos personalizados con los tipos de elementos de trabajo a los que aplican.

Módulo de Elementos de Trabajo

Este módulo gestiona la información relacionada con los elementos de trabajo, sus estados, responsables etc. Incluye las entidades:

- ***work_items***: Almacena la información de los elementos de trabajo, como su título, descripción, código, estimación, progreso, etc. Estos son los campos indispensables considerados el núcleo, ya que afectan a la forma de representar los nodos en el árbol de trabajo. Destaca una relación recursiva que permite representar la jerarquía de elementos de trabajo y subtareas.
- ***project_readmodels***: Almacena información de lectura de los proyectos para conocer a qué proyecto pertenece un elemento de trabajo.
- ***skill_readmodels***: Almacena información de lectura de las habilidades para conocer las habilidades requeridas por un elemento de trabajo.
- ***skill_work_items***: Tabla intermedia que asocia las habilidades con los elementos de trabajo a los que requieren con sus estadísticas calculadas.
- ***field_readmodels* y *field_type_readmodels***: Almacenan información de lectura de los campos personalizados y sus tipos para conocer los campos personalizados que tiene un elemento de trabajo.
- ***field_values***: Almacena los valores de los campos personalizados de cada elemento de trabajo.

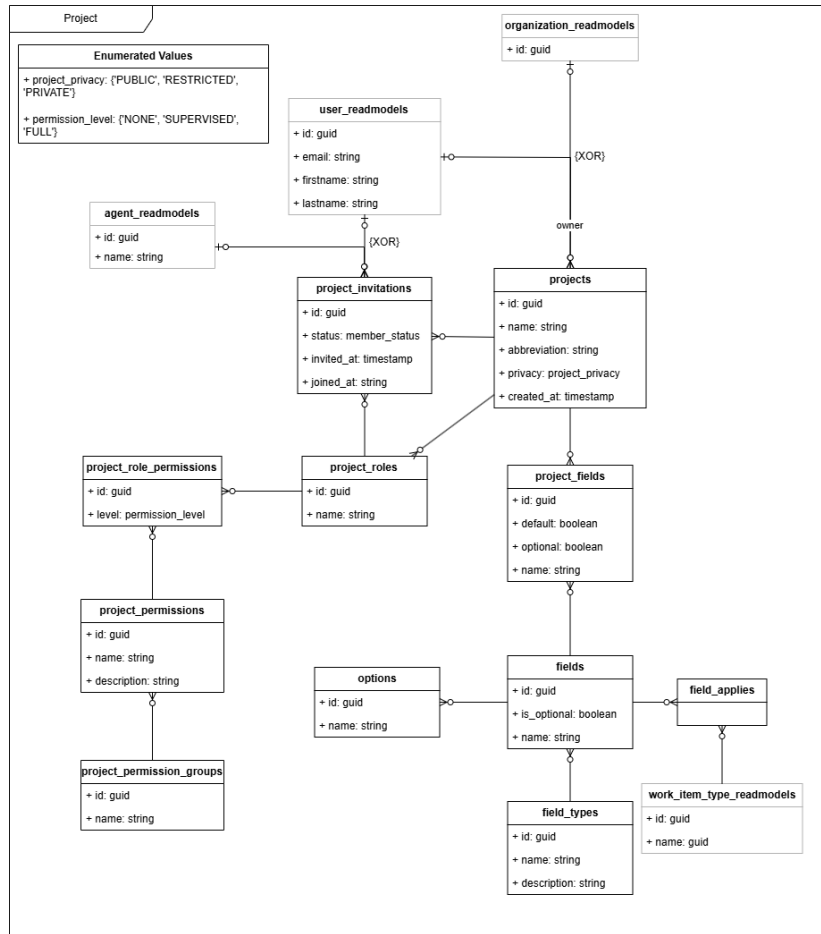


Figura 3.3: MER del módulo de Proyectos

- **work_item_status:** Almacena los estados de los elementos de trabajo definidos en el sistema.
- **work_item_types:** Almacena los tipos de elementos de trabajo definidos en el sistema.
- **user_readmodels:** Almacena información de lectura de los usuarios para conocer quién tiene asignado un elemento de trabajo y otros datos como quién hace comentarios o registra trabajo realizado.
- **attachments:** Almacena la información de los archivos adjuntos a los elementos de trabajo ya sea en su descripción o en los comentarios, incluyendo su URL y metadatos asociados.
- **work_item_comments:** Almacena los comentarios realizados en los elementos de trabajo, incluyendo el autor, el contenido y la fecha de creación.
- **work_logs:** Almacena los registros de trabajo realizados en los elementos de trabajo, incluyendo el usuario que realizó el registro, la cantidad de tiempo registrado y la fecha del registro.

- ***project_readmodels***: Almacena información de lectura de los proyectos para conocer a qué proyecto pertenece un agente.

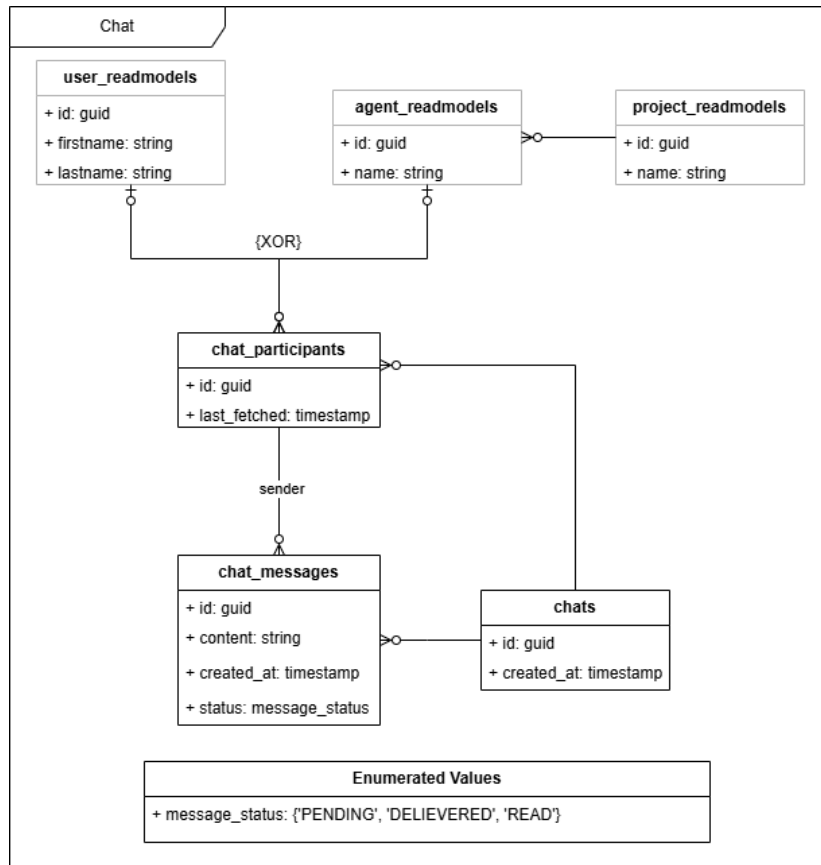


Figura 3.5: MER del módulo de Chats

Módulo de Agentes

Este módulo gestiona la información relacionada con los agentes de IA que colaboran en los proyectos. Incluye las entidades:

- ***project_readmodels***: Almacena información de lectura de los proyectos para conocer a qué proyecto pertenece un agente.
- ***model_providers***: Almacena los proveedores de modelos de IA disponibles en el sistema, como OpenAI, Claude, etc. junto con sus tokens de Interfaz de Programación de Aplicaciones (API).
- ***models***: Almacena los modelos de IA disponibles en el sistema, asociados a un proveedor de modelos con su descripción y precio por millón de tokens de entrada.
- ***agents***: Almacena la información de los agentes de IA creados en el sistema, incluyendo su nombre, instrucciones y *system prompt*.

- ***model_configs***: Tabla intermedia que asocia los agentes de IA con los modelos de IA que utilizan, incluyendo la configuración específica del modelo (temperatura y máximo de tokens).
- ***agent_templates***: Almacena las plantillas de agentes de IA predefinidos en el sistema, incluyendo su nombre, descripción e instrucciones. Ejemplos de plantillas pueden ser el agente *Daily Standup* o el agente *Desglose de Trabajo*.
- ***agent_time_triggers***: Almacena los disparadores temporales configurados para los agentes de IA, incluyendo la frecuencia de ejecución, zona horaria etc. Estos disparadores permiten configurar la ejecución automática de los agentes en función del tiempo, como por ejemplo ejecutar el agente *Daily Standup* todos los días a las 9:00 am.
- ***agent_event_triggers***: Almacena los disparadores de eventos configurados para los agentes de IA, incluyendo el tipo de evento que los dispara (creación de elemento de trabajo, cambio de estado, etc.) y las condiciones específicas del evento. Estos disparadores permiten configurar la ejecución automática de los agentes en función de eventos específicos dentro del proyecto.
- ***agent_executions***: Representa una instancia de ejecución de un agente de IA, que puede ser disparada mediante alguno de los disparadores definidos (mode: regular) o mediante un mensaje directo de un usuario (mode: direct query). Almacena información sobre la ejecución del agente, como su estado (pendiente, en progreso, completada, fallida y suspendida) y el *payload* de entrada.
- ***agent_execution_messages***: Almacena los mensajes generados durante la ejecución de un agente de IA, incluyendo los mensajes del sistema, los mensajes de usuario y los mensajes generados por el agente. Estos mensajes permiten mantener un contexto completo de la ejecución del agente.
- ***agent_execution_pending_events***: Almacena los eventos de los que está pendiente una ejecución de un agente de IA para poder reactivar la ejecución cuando se cumplan las condiciones del evento. Por ejemplo, si la ejecución de un agente está pendiente de la respuesta de un usuario para acabar su ejecución. Es importante no solo el tipo de evento esperado, sino el *correlation_id* del evento para poder asociarlo con la ejecución correcta.

Modelo de Notificaciones

Este módulo gestiona la información relacionada con las notificaciones enviadas a los usuarios, además de la aceptación de solicitudes de aprobación para acciones que requieren cierto nivel de permisos. Incluye las entidades:

- ***project_permission_readmodels***: Almacena información de lectura de los permisos de proyecto, para conocer los permisos de cada usuario en cada proyecto y poder determinar si una approval request le afecta o no.

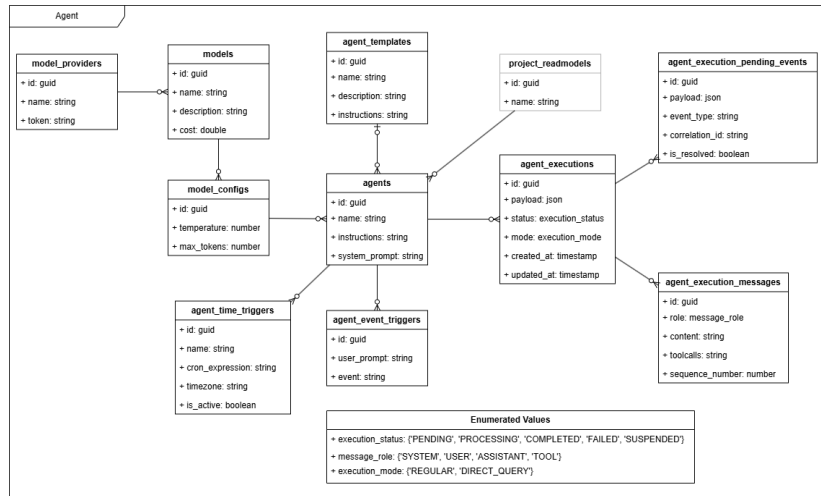


Figura 3.6: MER del módulo de Agentes

- ***organization_permission_readmodels***: Almacena información de lectura de los permisos de organización, para conocer los permisos de cada usuario en cada organización y poder determinar si una approval request le afecta o no.
- ***user_readmodels***: Almacena información de lectura de los usuarios para relacionarlos con las notificaciones que reciben, las que envían, las solicitudes de aprobación que les afectan, y sus comentarios en estas.
- ***notifications***: Almacena las notificaciones enviadas a los usuarios, incluyendo el tipo de notificación, el contexto (id de proyecto u organización), y el objetivo (id del elemento que genera la notificación, como un elemento de trabajo, un proyecto, un agente, etc.)
- ***approval_requests***: Almacena las solicitudes de aprobación generadas para acciones que requieren validación, incluyendo el estado, el tipo de contexto (proyecto u organización), el id del contexto, el id del elemento que genera la solicitud, el tipo de acción que se solicita aprobar (o nombre de la operación) y el payload con la información para realizar esa acción automáticamente si la solicitud es aprobada.
- ***actions***: Almacena las acciones que se pueden ejecutar automáticamente al aprobar una solicitud de aprobación.
- ***request_comments***: Almacena los comentarios realizados en las solicitudes de aprobación, incluyendo el autor, el contenido y la fecha de creación.

3. ANÁLISIS

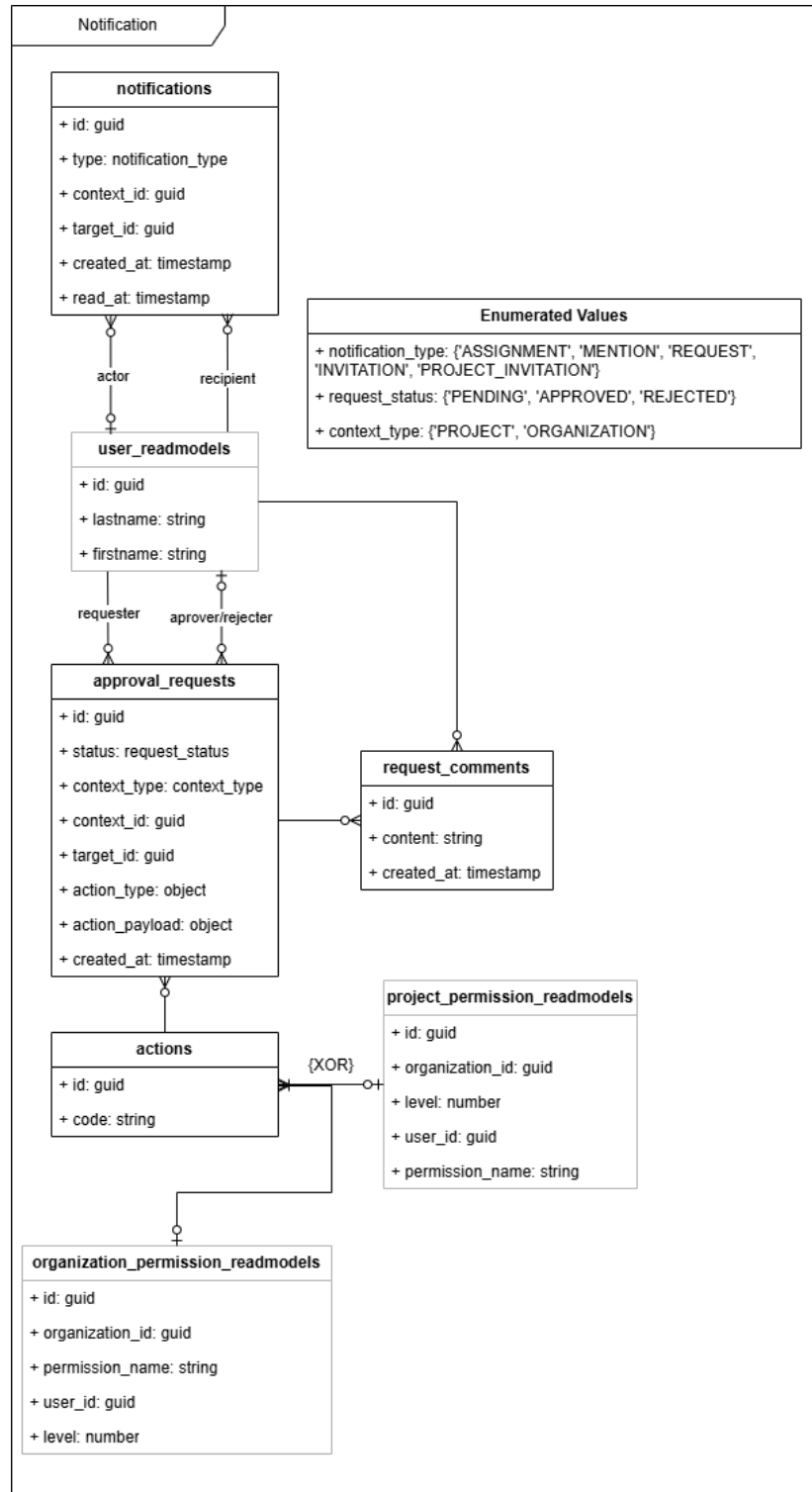


Figura 3.7: MER del módulo de Notificaciones

4.1 Arquitectura de la aplicación

Para representar la arquitectura de la aplicación, se ha recurrido a diagramas C4 [9], que permiten describir el sistema a diferentes niveles de abstracción, desde una visión general hasta los detalles de implementación. A continuación, se detallarán los tres primeros niveles de este modelo, llegando hasta el diagrama de componentes.

4.1.1 Diagrama de contexto (Nivel 1)

El nivel del diagrama de contexto proporciona una visión general del sistema, mostrando las interacciones entre el sistema, los usuarios y otros sistemas externos.

Los principales actores identificados en el diagrama de contexto son:

- **Usuario:** Actor humano que interactúa con la aplicación para gestionar su trabajo, colaborar en proyectos y comunicarse con otros usuarios y agentes.
- **Sistema:** Plataforma que proporciona las funcionalidades anteriormente descritas.
- **Proveedor de autenticación:** Servicio externo responsable de gestionar la autenticación de los usuarios (ej. Google).
- **Proveedores de IA:** Servicios externos que ofrecen modelos IA para procesar prompts del módulo de agentes (ej. OpenAI, Anthropic).

4.1.2 Diagrama de contenedores (Nivel 2)

Descendiendo al siguiente nivel de detalle, el diagrama de contenedores muestra las unidades de ejecución del sistema (aplicaciones, servicios, bases de datos), así como las responsabilidades de cada una y cómo se comunican entre sí.

Los contenedores identificados en el diagrama son:

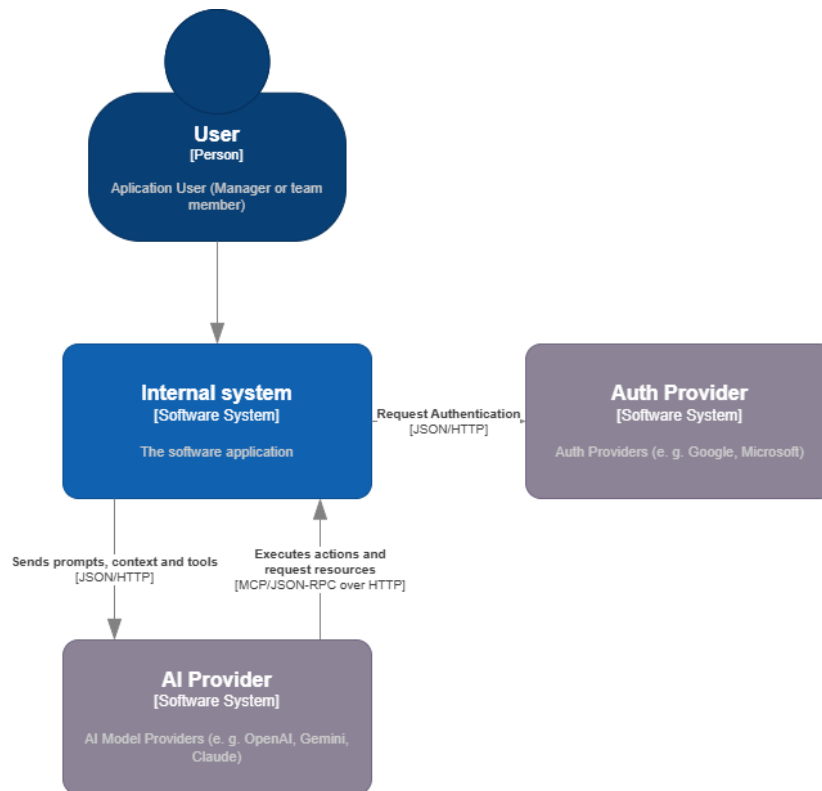


Figura 4.1: Diagrama de contexto de la aplicación

- **Frontend:** Aplicación web Single Page Application (SPA) ejecutada en el navegador del usuario, actúa como interfaz gráfica para interactuar con el sistema. Su responsabilidad es la composición visual de la interfaz, delegando la lógica de negocio al *backend* a través de llamadas a la API REST.
- **Backend:** Forma el núcleo del sistema, implementando la lógica de negocio, además de orquestar la comunicación entre diferentes servicios y gestionar la persistencia de datos. Expone una API REST para que el *frontend* pueda interactuar con él, y también se encarga de comunicarse con los proveedores externos (autenticación y IA).
- **Bases de datos:** Cumpliendo de forma estricta con los principios de DDD, cada módulo o *Bounded Context* tiene su propia base de datos, garantizando la independencia y autonomía de cada uno. Esto permite que cada módulo evolucione de forma aislada, sin afectar a los demás, y facilita la implementación de diferentes tecnologías de almacenamiento si fuera necesario. Esta separación anticipa una separación física futura en microservicios que se analiza más adelante al entrar al detalle del *backend*.
- **Almacenamiento de objetos:** Contenedor de infraestructura dedicado a almacenar archivos y recursos estáticos, como imágenes, documentos o cualquier otro tipo de archivo que deba ser accesible desde la aplicación. Se utiliza principalmente para almacenar los archivos adjuntos a los elementos de trabajo.

Este almacenamiento recibe lecturas y escrituras directamente desde el *frontend*, aliviando así la carga del *backend* y mejorando la escalabilidad del sistema, para mantener la seguridad, el acceso a este almacenamiento se gestiona mediante URLs firmadas que permiten un acceso temporal y controlado a los recursos almacenados.

- **Servidor de logs:** Contenedor de infraestructura encargado de centralizar y gestionar los logs generados por el sistema. Este contenedor recolecta, indexa y almacena los logs, trazas de ejecución y métricas de rendimiento producidos por la aplicación. Su uso permite monitorizar el estado del sistema, detectar errores y analizar el comportamiento de la aplicación, por lo que es una herramienta añadida únicamente para mejorar la mantenibilidad del sistema.

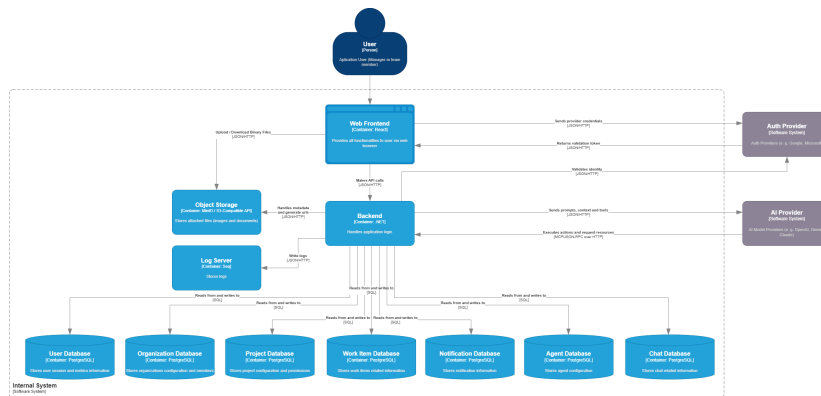


Figura 4.2: Diagrama de contenedores de la aplicación

4.1.3 Diagrama de componentes (Nivel 3)

Para diagramar a nivel de componentes, haremos un desglose en cada uno de los contenedores principales identificados en el nivel anterior.

Backend

En este nivel de detalle del *backend*, observamos ya una clara separación en módulos que se corresponden con los *Bounded Contexts* identificados en el análisis del dominio. La arquitectura adoptada para el sistema es la de un *monolito modular*. A diferencia de un monolito tradicional, donde la falta de fronteras estrictas tiende a generar un alto acoplamiento y dificulta el mantenimiento, y de una arquitectura pura de microservicios, que introduce una alta sobrecarga operativa y complejidad de red desde el primer día, el monolito modular ofrece un equilibrio pragmático [10]. Cada módulo encapsula su propia lógica de negocio y funciona de forma autónoma, lo que asegura un bajo acoplamiento y facilita enormemente una futura separación física en microservicios si las necesidades de escalabilidad del proyecto lo llegasen a requerir.

Para garantizar esta autonomía, la comunicación entre los distintos módulos se realiza a través de interfaces bien definidas mediante eventos y colas de mensajería asíncrona [11]. Esta independencia maximiza la escalabilidad lógica de cada contexto,

pero introduce complejidad en la gestión de los datos. Al prescindir de la transaccionalidad global e inmediata de las bases de datos centralizadas, el diseño del sistema debe afrontar las disyuntivas descritas por el Teorema CAP [12]. En este escenario, la aplicación prioriza la disponibilidad y el aislamiento de los módulos, asumiendo un modelo de *consistencia eventual*. Para manejar eficazmente esta separación entre la emisión de eventos y la actualización de los estados, el sistema se apoya en el patrón CQRS [8], optimizando de forma independiente los flujos de lectura y escritura.

El detalle técnico sobre cómo se orquesta esta arquitectura orientada a eventos de la comunicación asíncrona entre módulos se detallará más adelante en este mismo capítulo, donde se ilustrará el proceso apoyándose en un diagrama de secuencia explicativo.

Revisando ahora los componentes que encontramos en este diagrama, observamos:

- **Módulos:** Un componente referente a cada uno de los módulos identificados, que funcionan de forma independiente.
- **API Host:** Es el punto de entrada de la aplicación, se encarga de levantar el servidor web, gestionar las rutas y configurar cada uno de los módulos para que estén disponibles a través de la API REST.
- **Bus de eventos:** Componente central que facilita la comunicación asíncrona entre los módulos, permitiendo que cada uno emita eventos sin necesidad de conocer los consumidores de dichos eventos.
- **Background workers:** Componentes encargados de procesar tareas en segundo plano, como la sincronización de datos entre módulos u otros procesos que no requieren una respuesta inmediata al usuario.
- **Building blocks:** Componentes comunes que pueden ser utilizados por varios módulos, como abstracciones, funcionalidades transversales para realizar llamadas, gestionar conexiones con las bases de datos, publicación y suscripción de eventos, etc.

Backend (dentro de un módulo)

Entrando en el detalle de cada módulo, la arquitectura interna sigue los preceptos de la *Clean Architecture* [2], separando claramente las responsabilidades en capas concéntricas. En el núcleo se encuentran las entidades de dominio, que encapsulan la lógica de negocio pura y se mantienen independientes de cualquier tecnología o *framework*. Alrededor de estas entidades, en la capa de aplicación, se ubican los casos de uso, encargados de orquestar la lógica de negocio y coordinar las interacciones para cumplir con los requisitos funcionales.

La capa de infraestructura se ha estructurado adoptando la topología de la Arquitectura Hexagonal (también conocida como patrón de Puertos y Adaptadores) propuesta por Cockburn [13]. Con el añadido de que, esta capa de infraestructura se divide en dos: la *Driving Infrastructure*, que recibe las solicitudes entrantes, las traduce y las dirige hacia los casos de uso; y la *Driven Infrastructure* que se encarga de implementar las

4.1. Arquitectura de la aplicación

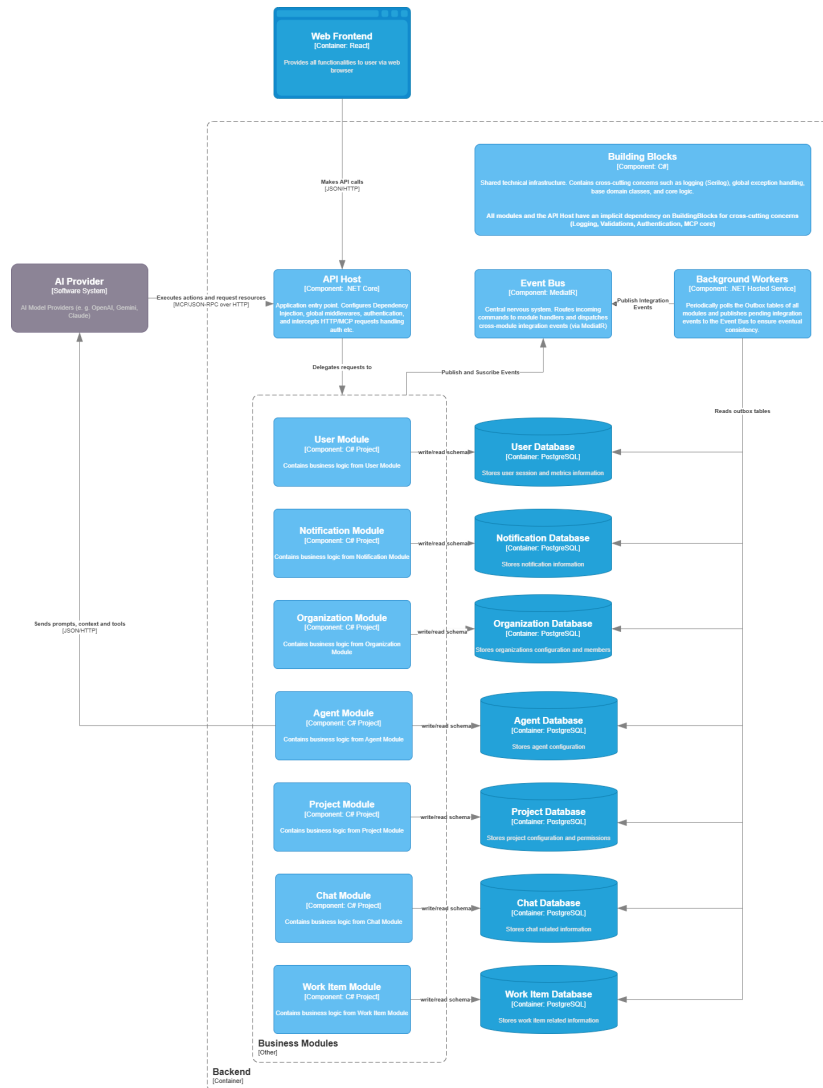


Figura 4.3: Diagrama de componentes del *backend*

interfaces definidas por el dominio para interactuar con servicios externos, tales como bases de datos o APIs de terceros.

Es importante destacar que toda esta estructura se vertebra en torno al Principio de Inversión de Dependencias [14]. La aplicación de este principio asegura que las dependencias a nivel de código fuente apunten hacia el centro del sistema, haciendo que el núcleo de la aplicación sea completamente independiente de las implementaciones tecnológicas concretas de las capas externas.

Además de estos componentes esenciales, cada módulo también incluye un componente de integración, que expone los eventos de integración que el módulo emite para que otros módulos puedan suscribirse a ellos.

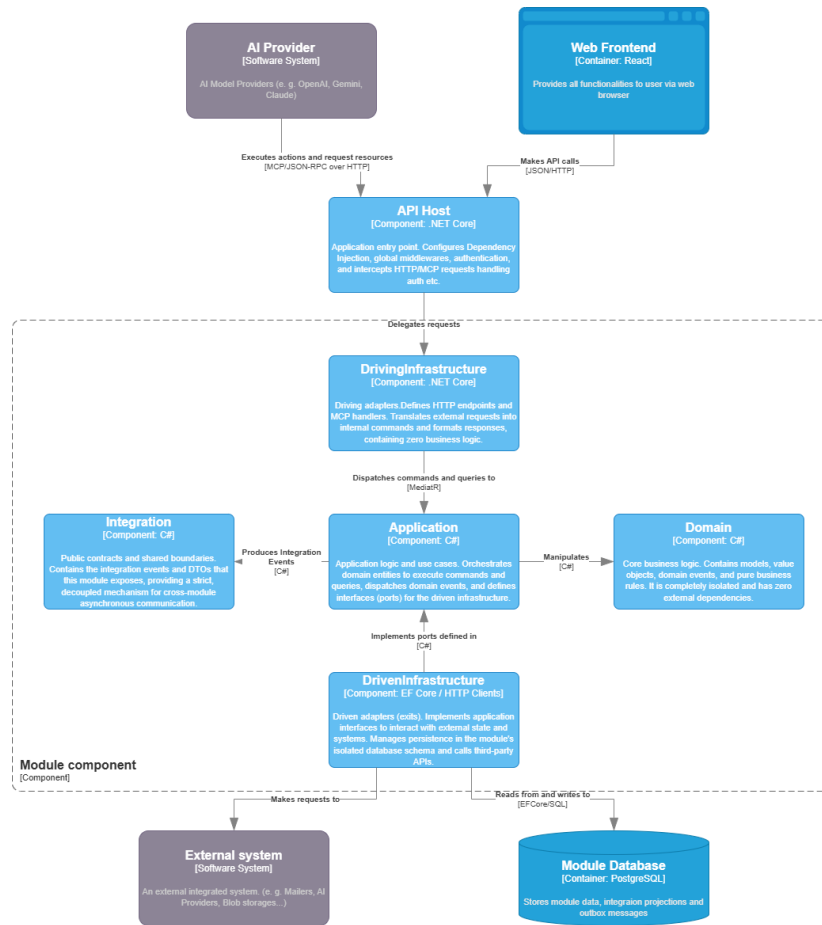


Figura 4.4: Diagrama de componentes dentro de un módulo

Frontend

El *frontend* de la plataforma se ha desarrollado como una aplicación web SPA utilizando React, lo que permite ofrecer una experiencia de usuario fluida y sin recargas de página. La arquitectura del cliente se aleja de los enfoques clásicos basados en el patrón Modelo-Vista-Controlador (MVC), para adoptar una arquitectura orientada a componentes que se alinea de forma natural con el ecosistema de React. Con el objetivo de mantener una alta cohesión y respetar las fronteras de los *Bounded Contexts* definidos en el dominio, se ha optado por estructurar la aplicación mediante *Vertical Slices* [15], priorizando el pragmatismo y la agrupación por funcionalidades en lugar de por capas técnicas.

Para lograr una correcta separación de responsabilidades dentro de cada módulo, la aplicación se estructura internamente en tres capas principales:

- **Capa de presentación:** Compuesta por los componentes visuales que conforman la interfaz de usuario. Utiliza de forma intensiva la metodología de Diseño Atómico (*Atomic Design*) [16] para garantizar la reutilización y la consistencia visual en toda la aplicación. A nivel organizativo, se divide en tres bloques: utilidades y componentes compartidos (*shared*) que pueden ser consumidos por cualquier módulo; componentes específicos de dominio encapsulados en su respectivo

módulo; y el componente *Root Composition*, encargado del enrutamiento global y la composición final de la interfaz.

- **Capa de proveedores:** Contiene los proveedores de la API de Contexto (*Context API*) de React que gestionan el estado global de la aplicación. Esta capa aísla y expone funcionalidades transversales críticas, tales como la autenticación, la gestión de sesiones de usuario y la internacionalización de la aplicación.
- **Capa de datos:** Encargada de abstraer y gestionar toda la comunicación asíncrona con el *backend* mediante llamadas a la API REST. Está formada por tres elementos principales: el cliente Protocolo de Transferencia de Hipertexto (HTTP) base; el cliente API, que actúa como un patrón *Facade* [17] exponiendo métodos tipados y específicos para cada uno de los *endpoints*; y finalmente el componente de *queries*, que utiliza la biblioteca React Query [18] para optimizar el ciclo de vida de las peticiones, gestionando de forma eficiente el estado asíncrono, la caché de datos y la invalidación de la misma.

4.2 Patrón de despliegue

En el contexto académico actual del proyecto, se ha tomado la decisión arquitectónica de establecer un entorno de despliegue pragmático y acotado. Desplegar una arquitectura distribuida en servicios *cloud* de alta disponibilidad excedería los objetivos, el presupuesto y el marco temporal de esta fase de desarrollo. Por tanto, el patrón de despliegue se centra en la contenerización para asegurar un entorno de integración local robusto e idéntico para todos los evaluadores, combinado con un alojamiento estático para el cliente web de Leaflet.

4.2.1 Orquestación con Docker Compose

Para evitar el clásico problema de 'funciona en mi máquina' y estandarizar la infraestructura necesaria para ejecutar el *backend*, se ha contenerizado la aplicación utilizando Docker y orquestando los servicios mediante Docker Compose.

Para el entorno de evaluación y producción, esta infraestructura se despliega íntegramente en un Servidor Privado Virtual (VPS) aprovisionado mediante los créditos académicos del programa *Azure for Students*. Esta decisión permite disponer de los recursos de cómputo necesarios a coste cero, automatizando las actualizaciones del servidor mediante un flujo de Integración y Despliegue Continuos (CI/CD) gestionado con GitHub Actions.

A través de un único archivo de configuración declarativa (`docker-compose.yml`), el sistema levanta simultáneamente todos los componentes necesarios aislados en su propia red virtual. Los contenedores orquestados son:

- **Backend:** El servidor principal que agrupa los módulos del monolito, empaquetado en una imagen Docker que contiene el entorno de ejecución. Expone la API REST por un puerto específico para comunicarse con el cliente web.
- **Bases de datos:** Los contenedores de las bases de datos relacionales asociadas a cada *Bounded Context*. Se despliegan con volúmenes persistentes asociados para evitar la pérdida de datos al destruir los contenedores.

- **Almacenamiento de objetos:** Contenedor de infraestructura (MinIO) que simula un entorno de *Object Storage*, utilizado para almacenar los archivos y adjuntos de forma desacoplada de las bases de datos.
- **Servidor de logs:** Contenedor encargado de centralizar y recolectar las trazas de ejecución y métricas generadas por el sistema.

4.2.2 Despliegue del cliente web

La ejecución y despliegue de la aplicación cliente (SPA desarrollada en React) se ha planteado en dos escenarios diferentes, separando el flujo de desarrollo del de producción:

- **Simulación en vivo (Entorno local):** Para las fases de diseño de interfaz (*UI*) y construcción de componentes, se utiliza el servidor de desarrollo propio del ecosistema. Esto habilita el *Hot Reloading*, permitiendo visualizar los cambios instantáneamente en el navegador del desarrollador sin el coste de tiempo que supone realizar un *build* de producción en cada iteración.
- **Despliegue automatizado en la nube (Vercel):** Para la puesta en producción, se ha descartado el uso del VPS interno en favor de **Vercel**, una plataforma Plataforma como Servicio (PaaS) optimizada específicamente para alojar aplicaciones *frontend* modernas. Esta plataforma se integra de forma nativa con el repositorio de código y proporciona un flujo de CI/CD completamente transparente: con cada actualización en la rama principal, Vercel ejecuta automáticamente el *build* de React, optimiza los archivos estáticos y los distribuye a través de una red Red de Distribución de Contenidos (CDN) global. Esto garantiza alta disponibilidad, despliegues sin intervención manual y acceso seguro a la plataforma a través de una URL pública a coste cero.

4.3 Diagramas de secuencia

Algunos de los procesos más críticos del sistema, como la sincronización de datos entre módulos, la ejecución de tareas de los agentes de IA o el flujo de autenticación, requieren de una orquestación más compleja que involucra múltiples componentes y servicios. Para ilustrar claramente cómo se desarrollan estos procesos, se han elaborado diagramas de secuencia Lenguaje de Modelado Unificado (UML) que detallan paso a paso las interacciones entre los diferentes elementos del sistema.

4.3.1 Sincronización de datos entre módulos

Como se mencionó en la definición de la arquitectura, la separación física y lógica de las bases de datos por cada *Bounded Context* impone el reto de mantener la consistencia de la información sin recurrir a transacciones distribuidas, las cuales penalizarían gravemente el rendimiento y la disponibilidad del sistema. Para resolver este desafío, se ha implementado una arquitectura basada en eventos combinada con el patrón *Transactional Outbox* y su contraparte *Inbox* [19].

Este patrón garantiza la entrega de mensajes de forma fiable (*at-least-once delivery*) y permite un procesamiento idempotente en el módulo destino, asumiendo un modelo de consistencia eventual. A continuación, se detalla el flujo de este proceso, tomando como ejemplo el cambio de estado de un elemento de trabajo y la consecuente generación de una notificación asíncrona (ver Figura 4.5):

1. **Fase 1: Actualización síncrona y Outbox:** El flujo comienza cuando el usuario solicita actualizar el estado de una tarea a través de la API. La capa de aplicación procesa el caso de uso, actualiza la entidad de dominio y genera un evento de integración (ej. *WorkItemStatusChangedEvent*). El aspecto crítico de esta fase es que la capa de infraestructura persiste tanto la actualización de la entidad de negocio como el evento (en una tabla auxiliar *OutboxMessages*) dentro de una única transacción ACID en la base de datos local del módulo. Esto asegura la consistencia interna: si la base de datos falla, ninguna de las dos operaciones se confirma. Inmediatamente después, se devuelve una respuesta exitosa HTTP 200 al usuario, sin bloquear el hilo principal esperando la comunicación con otros módulos.
2. **Fase 2: Despacho de eventos (Outbox Webjob):** Un proceso en segundo plano (*background job*) aislado del flujo web consulta periódicamente la tabla *OutboxMessages* en busca de eventos no publicados. Al encontrarlos, los extrae, los publica en el bus de integración (*Event Bus*) y actualiza su estado a procesado. Esta separación permite aislar la latencia o posibles caídas del sistema de mensajería (tras un futuro cambio a microservicios) del tiempo de respuesta de la API.
3. **Fase 3: Recepción aislada (Inbox):** El bus de eventos entrega el mensaje al módulo suscriptor (en este diagrama, el módulo de Notificaciones). En lugar de ejecutar la lógica de negocio inmediatamente, el suscriptor se limita a guardar el evento recibido en su propia tabla *InboxMessages*. Esta operación de persistencia está diseñada mediante restricciones de base de datos para ser idempotente, protegiendo al sistema receptor de posibles entregas duplicadas por parte del bus de eventos.
4. **Fase 4: Procesamiento asíncrono (Inbox Job):** Finalmente, una tarea programada (*webjob*) dentro del módulo de Notificaciones lee los mensajes pendientes de su bandeja de entrada por lotes (*batches*). Por cada evento, ejecuta la lógica de aplicación correspondiente, como generar la entidad de Notificación para el usuario asignado, la persiste en su base de datos y marca el mensaje original del *Inbox* como procesado.

4.3.2 Ejecución de tareas de los agentes de IA

A diferencia de las interacciones síncronas tradicionales mediante *prompts*, los agentes de IA en la plataforma operan como miembros dentro del equipo. Esto implica que sus tareas pueden ser de larga duración (*long-running workflows*) y requieren la capacidad de reaccionar a eventos, ejecutar herramientas del sistema y, cuando es necesario, pausar su ejecución para esperar la validación o respuesta de un usuario humano.

4. DISEÑO

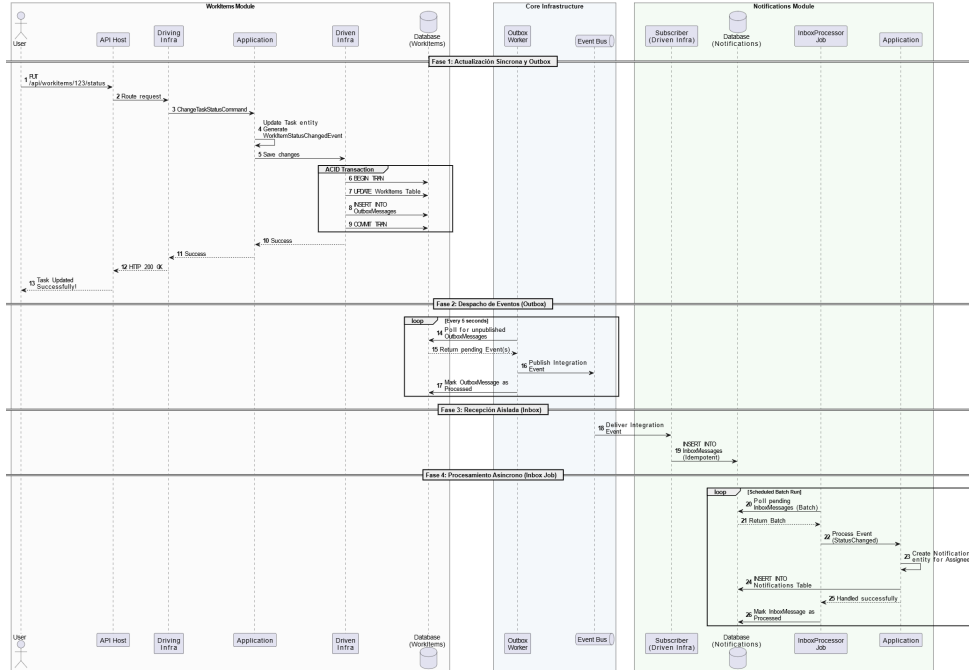


Figura 4.5: Diagrama de secuencia de la sincronización de datos mediante el patrón Outbox/Inbox.

Para modelar este comportamiento asíncrono y reactivo de forma resiliente, se ha diseñado un motor de ejecución propio (*Agent Core*). La Figura 4.6 ilustra el ciclo de vida completo de la ejecución de una tarea por parte de un agente, dividido en cuatro fases principales:

1. **Fase 1: Desencadenador (*Trigger*):** La activación de un agente puede producirse por dos vías. Por un lado, mediante *Event Triggers*: cuando ocurre un evento de dominio en el bus (por ejemplo, la creación de un nuevo elemento de trabajo), el manejador consulta en base de datos si existe algún agente suscrito a dicho evento y, de ser así, inserta un registro de ejecución en estado pendiente. Por otro lado, mediante *Time Triggers*: el planificador (Quartz) dispara eventos basados en expresiones *cron* (por ejemplo, para generar el *Daily Standup* cada mañana), generando igualmente una ejecución pendiente.
2. **Fase 2: Ejecución e Invocación del Modelo:** Un proceso en segundo plano (*web-job*) recoge las ejecuciones pendientes. El sistema compone el contexto, inyectando el *system prompt* del agente, la información del entorno y el historial de ejecución, y realiza la llamada al Modelo de Lenguaje Grande (LLM). A partir de aquí, la IA razona sobre los pasos a seguir [20]. Si decide ejecutar una acción directa, invoca la API de *Tools*, procesa el resultado y continúa. Sin embargo, si el agente determina que necesita información o aprobación de un humano, invoca la herramienta *SuspendWorkflow*. Esta acción actualiza el estado de la ejecución a suspendida, guarda el contexto de la conversación para no perder memoria de la misma, e inserta un registro indicando qué evento específico está esperando para reactivarse.

3. **Fase 3: Reactivación por Evento:** Mientras el flujo está suspendido, el agente no consume recursos computacionales. Si el sistema detecta el evento esperado en el bus (por ejemplo, un usuario responde al comentario del agente), el manejador localiza la ejecución suspendida correspondiente mediante identificadores de correlación, la devuelve al estado pendiente y el flujo retorna automáticamente a la Fase 2, permitiendo que el LLM continúe su razonamiento con la nueva información aportada por el usuario.
4. **Fase 4: Reactivación por Timeout:** Para evitar que los flujos de trabajo queden bloqueados indefinidamente si el usuario humano ignora la solicitud del agente, el sistema cuenta con un mecanismo de caducidad. Si se alcanza el tiempo máximo de espera definido, el planificador (Quartz) reactiva la ejecución de forma forzada. En este caso, se inyecta un *prompt* al modelo indicando explícitamente que el tiempo de espera ha expirado sin respuesta del usuario. Basándose en esta instrucción, el LLM puede decidir sus siguientes pasos de forma autónoma, como escalar el problema a un gestor, reintentar la comunicación o continuar con la ejecución asumiendo un escenario por defecto.

4.3.3 Flujo de autenticación

La gestión de la identidad y las sesiones en aplicaciones web modernas (SPA) requiere un diseño riguroso para prevenir vulnerabilidades de seguridad, tales como el robo de credenciales mediante ataques Cross-Site Scripting (XSS) o Cross-Site Request Forgery (CSRF). Para garantizar el acceso seguro a la aplicación, se ha implementado un flujo de autenticación delegada basado en OAuth 2.0 [1] (concretamente mediante Google Single Sign-On (SSO)), combinado con un sistema de doble token (*Access Token* y *Refresh Token*) con rotación de credenciales.

El proceso completo, ilustrado en la Figura 4.7, se divide en dos fases operativas:

1. **Fase 1: Autenticación inicial y aprovisionamiento (Google SSO):** El ciclo comienza cuando el usuario inicia sesión mediante el proveedor de identidad externo (Google). El cliente web recibe un token de autorización temporal y lo transmite al *backend*. Por motivos de seguridad, el servidor no confía en la aserción del cliente y verifica criptográficamente la validez de este token directamente contra los servidores de Google. Una vez validada la identidad (nombre y correo electrónico), el sistema comprueba si el usuario existe en la base de datos de la plataforma; de no ser así, se le registra automáticamente.

A continuación, el *backend* genera dos credenciales de sesión:

- Un *Access Token* (en formato JSON Web Token (JWT) [21]) de vida corta (60 minutos), que se devuelve en el cuerpo de la respuesta HTTP. El *frontend* almacena este token **exclusivamente en memoria** (nunca en *Local Storage*), protegiéndolo contra extracciones maliciosas por parte de *scripts* de terceros.
- Un *Refresh Token* de larga duración (7 días), el cual se persiste en la base de datos y se envía al cliente empaquetado en una cabecera Set-Cookie

4. DISEÑO

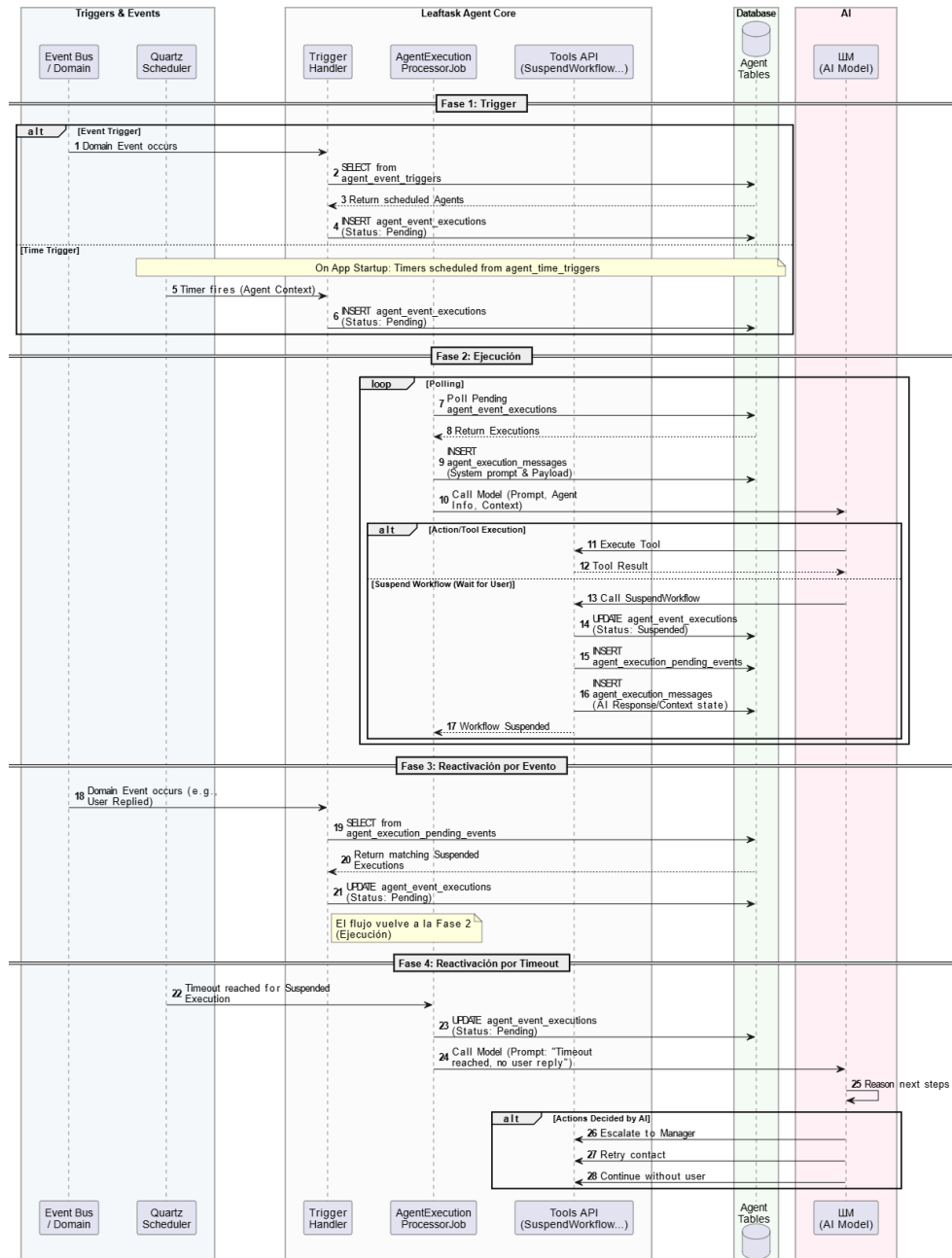


Figura 4.6: Diagrama de secuencia del ciclo de ejecución, suspensión y reactivación de un agente de IA.

con los atributos `HttpOnly` y `Secure`. Esto garantiza que el navegador lo almacene de forma segura e inaccesible mediante JavaScript.

2. **Fase 2: Flujo de renovación de sesión (*Refresh Flow*):** Dado que el *Access Token* expira rápidamente, el sistema debe ser capaz de renovarlo sin interrumpir la experiencia del usuario. Cuando el *frontend* intenta consumir un recurso protegido y recibe un error HTTP 401 *Unauthorized* (indicando que el JWT ha caducado o no existe), un interceptor en el cliente HTTP pausa la petición original y lanza una llamada al *endpoint* de renovación (`/session/refresh`).

Al realizar esta petición, el navegador incluye automáticamente la cookie `HttpOnly` con el *Refresh Token*. El *backend* recibe la petición, valida la existencia y vigencia del token en su base de datos, e implementa un mecanismo de **Rotación de Tokens de Refresco**:

- Si el token no es válido o ha caducado, se deniega la renovación y el usuario es redirigido a la pantalla de inicio de sesión.
- Si el token es válido, el sistema revoca inmediatamente ese *Refresh Token* antiguo de la base de datos, generando un nuevo par de *Access Token* y *Refresh Token*. Este mecanismo asegura que si un token de refresco es interceptado, su reutilización invalidará la familia de tokens, alertando de una brecha de seguridad.

Finalmente, el *frontend* actualiza su JWT en memoria y reintenta de forma transparente la petición original que había fallado, completando así el proceso sin fricción para el usuario.

4.4 Patrones de diseño

Durante el desarrollo del código fuente, se ha aplicado de forma sistemática un conjunto de patrones de diseño orientados a objetos. Muchos de estos son los patrones clásicos de diseño de software [17], proporcionan soluciones probadas y estandarizadas a problemas recurrentes de diseño de software. Su aplicación no solo resuelve retos técnicos de bajo nivel, sino que establece un vocabulario común y facilita la mantenibilidad, extensibilidad y legibilidad del código.

Con el fin de estructurar su análisis, los patrones implementados se han clasificado en tres categorías fundamentales, de acuerdo con su propósito principal dentro de la arquitectura del sistema: patrones creacionales, estructurales y de comportamiento.

4.4.1 Patrones creacionales

Los patrones creacionales abstraen el proceso de instanciación de los objetos, ocultando la lógica de creación. En el contexto de la aplicación, se han aplicado diversas variantes de estos patrones adaptadas tanto al paradigma orientado a objetos del *backend* como al paradigma funcional del *frontend*.

4. DISEÑO

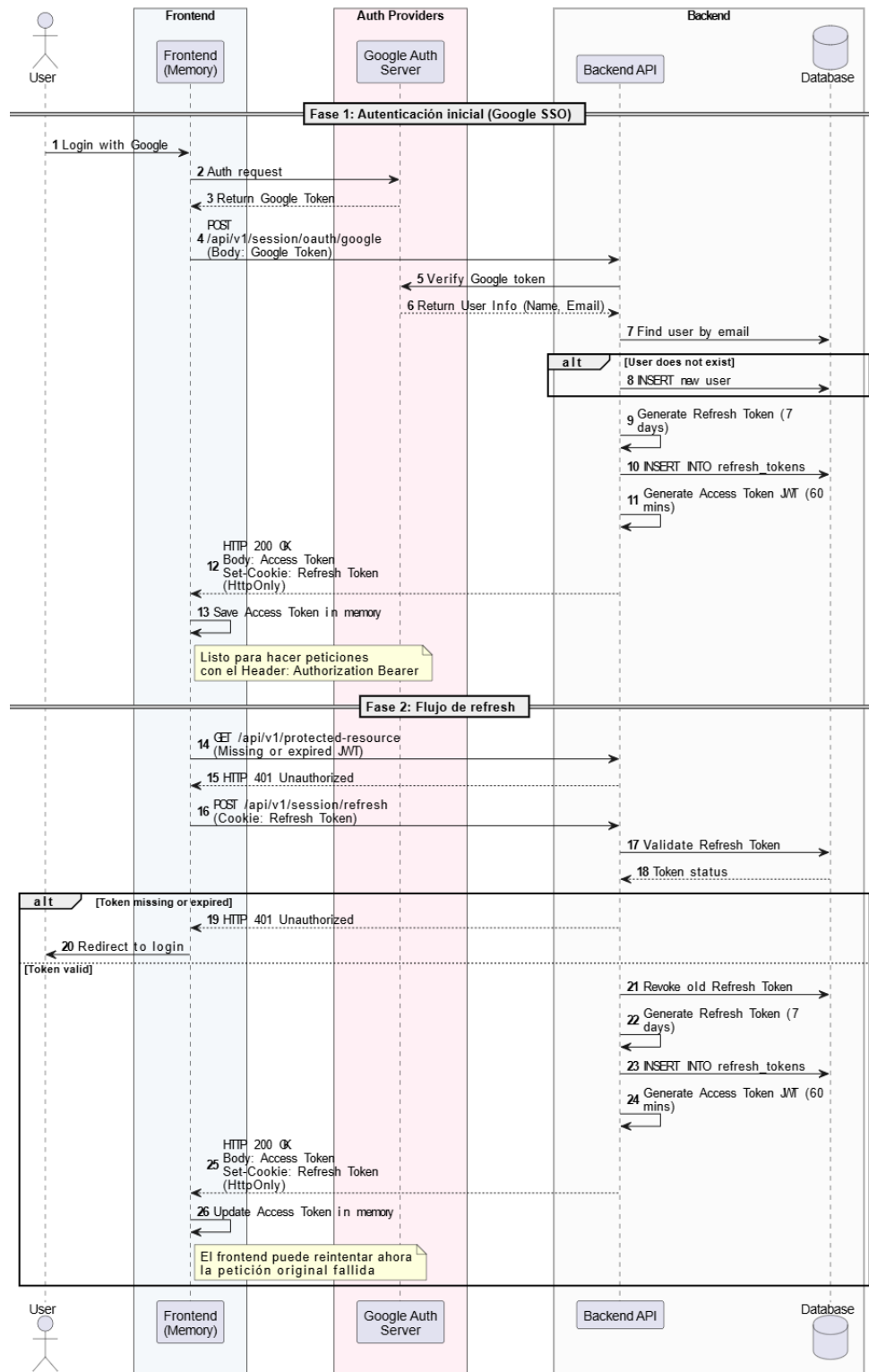


Figura 4.7: Diagrama de secuencia del flujo de autenticación, aprovisionamiento de usuarios y rotación de tokens.

Static Factory Method (Agregados de Dominio)

En el núcleo de la aplicación, siguiendo los principios del DDD, se ha descartado el uso de constructores públicos para las Entidades y *Aggregate Roots*. En su lugar, se ha implementado el patrón *Static Factory Method*.

Este patrón define métodos estáticos de creación (Create) que actúan como el único punto de entrada válido para instanciar un objeto. Al mantener el constructor privado, se garantiza que no se puedan crear entidades en un estado inválido, permitiendo encapsular la validación de invariantes de negocio y la emisión del evento de dominio inicial en una única transacción atómica en memoria.

```

1 public sealed class User : Entity
2 {
3     // Constructor privado: impide la instanciación directa y
4     // anula estados inválidos
5     private User(Guid id, string firstName, string lastName,
6     string email) { ... }
7
8     // Factory Method estático: Valida invariantes y emite el
9     // evento de dominio
10    public static User Create(string firstName, string
11    lastName, string email)
12    {
13        User user = new(Guid.NewGuid(), firstName, lastName,
14        email);
15        user.Raise(new UserCreatedDomainEvent(
16            user.Id, user.FirstName, user.LastName, user.
17            Email));
18        return user;
19    }
20 }

```

Listing 4.1: Implementación del patrón Static Factory Method en la entidad User.

Factory (Desacoplamiento de Políticas)

Para la creación de objetos cuya inicialización depende de configuraciones externas o servicios de infraestructura, se ha aplicado el patrón *Factory* clásico mediante clases dedicadas. Este enfoque separa la responsabilidad de crear un objeto de la lógica que decide cómo debe configurarse.

Un ejemplo claro en la plataforma es *RefreshTokenFactory*. Esta fábrica centraliza la construcción de tokens de sesión, pero delega la política de expiración a una interfaz inyectada (*IRefreshTokenExpirationPolicy*). Gracias a esta inversión de dependencias, la duración de los tokens puede variar dinámicamente (por ejemplo, según el entorno de despliegue o regulaciones de seguridad) sin necesidad de modificar la fábrica ni la lógica de negocio.

```

1 public class RefreshTokenFactory(
2     IRefreshTokenExpirationPolicy policy) :
3     IRefreshTokenFactory

```

```

2 {
3     public RefreshToken CreateForUser(Guid userId)
4     {
5         // La fabrica obtiene la politica de expiracion
        inyectada dinamicamente
6         TimeSpan duration = policy.ExpirationDuration;
7         return RefreshToken.Create(userId, duration);
8     }
9 }

```

Listing 4.2: Implementación del patrón Factory delegando logicas de expiracion.

Factory Function (Adaptadores de UI en Frontend)

En el entorno del *frontend*, dominado por React y el paradigma de la programación funcional, el concepto de fábrica se ha adaptado utilizando Funciones de Orden Superior (*Higher-Order Functions*).

El patrón *Factory Function* se utiliza para resolver la necesidad de crear mapeadores de datos que requieren dependencias previas (como diccionarios o catálogos). La función `makeProjectNodeAdapter` actúa como una fábrica: recibe los catálogos de tipos y estados una sola vez y retorna una nueva función pura. Esta función resultante (`WorkItemData → NodeVisual`) puede ser reutilizada de forma masiva y eficiente por la capa de presentación para transformar los datos del servidor en nodos visuales para el Árbol de Trabajo.

```

1 export const makeProjectNodeAdapter = (
2   types: WorkItemTypeData[],
3   statuses: WorkItemStatusData[]) => {
4
5   // Retorna una funcion pura pre-configurada gracias al
        closure
6   return (raw: WorkItemData): NodeVisual => {
7     const typeName = types.find((t) => t.id === raw.typeId)?.
        name ?? 'Task';
8     const color = STATUS_COLORS[
9       statuses.find((s) => s.id === raw.statusId)?.name ??
10      ], ?? DEFAULT_COLOR;
11
12     return {
13       title: raw.code,
14       subtitle: raw.title,
15       color,
16       shape: typeName === 'Bug' ? 'spike' : 'circle',
17       // ...
18     };
19   };
20 }

```

Listing 4.3: Implementación de Factory Function mediante clausuras en TypeScript.

4.4.2 Patrones estructurales

Los patrones estructurales se centran en cómo las clases y objetos se componen para formar estructuras más grandes y complejas, garantizando que el sistema sea flexible y eficiente.

Decorator (Comportamientos de canalización con MediatR)

El patrón *Decorator* permite añadir responsabilidades dinámicamente a un objeto envolviéndolo en otro objeto con la misma interfaz, sin alterar su código fuente. En el *backend*, este patrón se ha materializado a través de los *Pipeline Behaviors* de la biblioteca MediatR.

La interfaz `IPipelineBehavior<TRequest, TResponse>` permite encadenar capas de ejecución alrededor de cada manejador (*handler*) de casos de uso. De este modo, se ejecutan operaciones transversales de forma secuencial: primero la trazabilidad y medición de tiempos (*Logging*), seguido de la validación estructural (*FluentValidation*) y, finalmente, la autorización. Cada comportamiento cede el control a la siguiente capa mediante la invocación de `next()`, manteniendo a los casos de uso completamente agnósticos respecto a estas validaciones externas.

```

1 // Logging: mide y traza todas las peticiones
2 public sealed class LoggingBehavior<TRequest, TResponse>(
3     ILogger<...> logger)
4     : IPipelineBehavior<TRequest, TResponse>
5 {
6     public async Task<TResponse> Handle(TRequest request,
7     RequestHandlerDelegate<TResponse> next, CancellationToken
8     ct)
9     {
10         logger.LogInformation("[{Behavior}] Executing: {Name}",
11         nameof(LoggingBehavior<,>), typeof(TRequest).Name);
12
13         Stopwatch timer = Stopwatch.StartNew();
14         TResponse response = await next(ct);
15
16         logger.LogInformation("[{Behavior}] Completed in {Ms} ms",
17         nameof(LoggingBehavior<,>), timer.ElapsedMilliseconds);
18         return response;
19     }
20 }
21
22 // Validacion: ejecuta todos los IValidator<TRequest>
23 // registrados
24 public sealed class ValidationBehavior<TRequest, TResponse>(
25     IEnumerable<IValidator<TRequest>> validators)
26     : IPipelineBehavior<TRequest, TResponse>
27 {

```

```

24     public async Task<TResponse> Handle(TRequest request,
    RequestHandlerDelegate<TResponse> next, CancellationToken
    ct)
25     {
26         if (!validators.Any()) return await next(ct);
27
28         ValidationResult[] results = await Task.WhenAll(
29             validators.Select(v => v.ValidateAsync(new(
    request), ct)));
30
31         List<ValidationFailure> errors = results
32             .Where(r => !r.IsValid)
33             .SelectMany(r => r.Errors).ToList();
34
35         if (errors.Count > 0) throw new ValidationException(
    errors);
36         return await next(ct);
37     }
38 }

```

Listing 4.4: Implementación del patrón Decorator mediante Pipeline Behaviors para trazabilidad y validación.

Proxy (Interceptores HTTP en Frontend)

El patrón *Proxy* proporciona un intermediario o sustituto para controlar el acceso a un objeto real, añadiendo comportamiento de forma transparente. En la aplicación cliente, los interceptores de la biblioteca Axios actúan como un *proxy* bidireccional para el cliente HTTP.

El interceptor de peticiones (*request*) inyecta la cabecera de autorización (JWT) en todas las llamadas salientes, liberando a los servicios individuales de esta responsabilidad. Por su parte, el interceptor de respuestas (*response*) actúa como un mecanismo de resiliencia: si detecta un error 401 *Unauthorized*, pausa las llamadas, ejecuta el refresco del token de forma centralizada (utilizando una promesa compartida para evitar condiciones de carrera ante múltiples peticiones simultáneas) y, una vez obtenido el nuevo token, reintenta automáticamente la petición original de forma invisible para el usuario.

```

1  // Request: inyecta el token de acceso en cada petición
2  apiClient.interceptors.request.use((config) => {
3      const requestConfig = config as RetriableRequestConfig;
4      if (!requestConfig.skipAuthorization) {
5          const accessToken = getAccessToken();
6          if (accessToken) {
7              requestConfig.headers.set('Authorization', 'Bearer ${
    accessToken}');
8          }
9      }
10     return requestConfig;
11 });
12

```

```

13 // Response: detecta 401, refresca y reintenta (una sola vez)
14 apiClient.interceptors.response.use(
15   (response) => response,
16   async (error: AxiosError) => {
17     const requestConfig = error.config as
18     RetriableRequestConfig | undefined;
19     if (!requestConfig || requestConfig._retry || error.
20     response?.status !== 401) {
21       return Promise.reject(error);
22     }
23
24     requestConfig._retry = true;
25     // Promesa compartida para sincronizar multiples
26     // peticiones fallidas
27     const refreshedToken = await getRefreshTokenOnce();
28     requestConfig.headers.set('Authorization', 'Bearer ${
29     refreshedToken}');
30
31     return apiClient(requestConfig);
32   }
33 );

```

Listing 4.5: Implementación del patrón Proxy mediante interceptores HTTP para inyección y rotación de tokens.

Facade (ApiGateway de cliente)

El patrón *Facade* ofrece una interfaz unificada, simplificada y de alto nivel que oculta la complejidad de un subsistema subyacente compuesto por múltiples interfaces.

En la capa de datos del *frontend*, *ApiGateway* implementa este patrón agrupando todos los controladores (*Gateways*) individuales bajo un único espacio de nombres jerárquico. De este modo, los componentes de Interfaz de Usuario (UI) y los gestores de estado asíncrono no necesitan importar docenas de archivos ni conocer la topología de la API, simplemente importan el objeto unificado y navegan por sus propiedades de forma semántica e intuitiva, mejorando significativamente la mantenibilidad y la experiencia del desarrollador (*Developer Experience*).

```

1 export const ApiGateway = {
2   organization: {
3     invitations: OrganizationInvitationsGateway,
4     management: OrganizationManagementGateway,
5     members: OrganizationMembersGateway,
6     roles: OrganizationRolesGateway,
7   },
8   project: {
9     customFields: ProjectCustomFieldsGateway,
10    management: ProjectManagementGateway,
11  },
12  workItem: {
13    management: WorkItemsGateway,
14    workLogs: WorkLogGateway,
15    attachments: AttachmentGateway,

```

```

16     comments:    CommentGateway,
17 },
18 user:  { session: SessionGateway, users: UsersGateway },
19 agent: AgentGateway,
20 chat:  ChatGateway,
21 } as const;
22
23 // Uso simplificado desde cualquier componente:
24 // ApiGateway.workItem.management.createWorkItem(projectId,
    payload)

```

Listing 4.6: Implementación del patrón Facade para agrupar los servicios de llamadas HTTP.

Specification (Encapsulamiento de Consultas)

El patrón *Specification*, derivado de las prácticas de DDD, encapsula un criterio de consulta y las reglas de negocio asociadas en un objeto tipado, reutilizable y fácilmente testeable.

En lugar de dispersar sentencias de filtrado (Where) y carga ansiosa (Include) a lo largo de los repositorios o manejadores, se define una clase base `BaseSpecification<T>` que establece el contrato. Las especificaciones concretas, como `UserWithActiveRefreshTokensSpecification` heredan de esta base y configuran semánticamente las expresiones condicionales necesarias. Esto garantiza el principio de Responsabilidad Única y permite construir consultas compuestas de alta complejidad de forma declarativa y segura ante refactorizaciones.

```

1 public abstract class BaseSpecification<T>(Expression<Func<T,
    bool>> criteria)
2     : ISpecification<T>
3 {
4     private readonly List<Expression<Func<T, object>>>
        _includes = [];
5     public Expression<Func<T, bool>> Criteria { get; } =
        criteria;
6     public IReadOnlyCollection<Expression<Func<T, object>>>
        Includes
7         => _includes.AsReadOnly();
8
9     protected void AddInclude(Expression<Func<T, object>>
        expr)
10         => _includes.Add(expr);
11 }
12
13 // Especificacion concreta que encapsula la logica de
    filtrado e inclusion
14 public sealed class UserWithActiveRefreshTokensSpecification
15     : BaseSpecification<User>
16 {
17     public UserWithActiveRefreshTokensSpecification(Guid
        userId)

```



```

18         : base(user => user.Id == userId)
19     {
20         AddInclude(user => user.RefreshTokens
21             .Where(rt => rt.RevokedAt == null && rt.ExpiresAt
22                 > DateTime.UtcNow));
23     }

```

Listing 4.7: Implementación del patrón Specification para la capa de persistencia de datos.

4.4.3 Patrones de comportamiento

Los patrones de comportamiento se centran en los algoritmos y en la asignación de responsabilidades entre los objetos. En lugar de enfocarse únicamente en la estructura, estos patrones gestionan el flujo de control y la comunicación dinámica en el sistema, permitiendo que los componentes interactúen de forma eficiente y con un bajo nivel de acoplamiento.

CQRS (Separación de Responsabilidades)

El patrón CQRS [8] separa estrictamente las operaciones que modifican el estado del sistema (*Commands*) de aquellas que únicamente leen información (*Queries*). En el *backend*, esta separación permite optimizar y escalar ambas vías de forma independiente. A nivel de implementación, las interfaces *ICommand* e *IQuery<T>* actúan como marcadores que extienden *IRequest* de MediatR, garantizando mediante el sistema de tipos estático que cada manejador reciba exclusivamente su tipo correspondiente.

```

1 public interface ICommand : IRequest;
2 public interface ICommand<out TResponse> : IRequest<TResponse>;
3 public interface IQuery<out TResponse> : IRequest<TResponse>;
4
5 // Command: modifica el estado del dominio
6 public sealed record CreateOrganizationCommand(
7     string Name, string Description, string Website)
8     : ICommand<Result<OrganizationDto>>;
9
10 // Query: operacion de solo lectura
11 public sealed record GetMyOrganizationsQuery(
12     int Limit, string? Cursor, IReadOnlyCollection<string>
13     Sort)
14     : IQuery<Result<PaginatedResult<SimpleOrganizationDto>>>;

```

Listing 4.8: Definición de interfaces base para CQRS y su aplicación práctica.

Mediator (Desacoplamiento de Flujos)

El patrón *Mediator* centraliza la comunicación entre objetos para evitar dependencias directas entre ellos. En la arquitectura del servidor, la biblioteca MediatR actúa como

el mediador central: los controladores API, los manejadores de eventos y las tareas en segundo plano envían peticiones a través de las interfaces *ISender* o *IPublisher*, ignorando por completo qué clase concreta procesará la solicitud.

```

1 // Controller base que inyecta el ISender del Mediator
2 public abstract class ApiBaseController : ControllerBase
3 {
4     private ISender? _sender;
5     protected ISender Sender => _sender ??=
6         HttpContext.RequestServices.GetRequiredService<
7     ISender>();
8 }
9 // Controller concreto enviando un Command
10 [HttpPost]
11 public async Task<IActionResult> Create(
12     [FromBody] CreateOrganizationRequest request,
13     CancellationToken ct)
14 {
15     var command = new CreateOrganizationCommand(
16         request.Name, request.Description, request.Website);
17     return HandleResult(await Sender.Send(command, ct));
18 }

```

Listing 4.9: Uso del patrón Mediator para desacoplar controladores de sus manejadores.

Repository (Abstracción de Persistencia)

Aunque catalogado fundacionalmente como un patrón de dominio [5], el *Repository* actúa conductualmente como una abstracción de colección en memoria, ocultando los detalles de acceso a datos. Cada módulo del sistema define las interfaces de sus repositorios en la capa de dominio, mientras que su implementación concreta (basada en *Entity Framework Core*) reside en la capa de infraestructura, cumpliendo el Principio de Inversión de Dependencias.

```

1 // Dominio: contrato agnostico a la infraestructura
2 public interface IOrganizationRepository
3 {
4     Task<Organization?> GetByIdAsync(Guid id,
5     CancellationToken ct = default);
6     Task AddAsync(Organization org, CancellationToken ct =
7     default);
8     Task SaveChangesAsync(CancellationToken ct = default);
9 }
10 // Infraestructura: implementacion concreta acoplada a EF
11 // Core
12 public class OrganizationRepository(OrganizationDbContext
13     dbContext)
14     : IOrganizationRepository
15 {
16     public async Task<Organization?> GetByIdAsync(Guid id,
17     CancellationToken ct = default) =>

```

```

14         await dbContext.Organizations
15             .AsSplitQuery()
16             .Include(o => o.Invitations)
17             .Include(o => o.Roles).ThenInclude(r => r.
Permissions)
18             .FirstOrDefaultAsync(o => o.Id == id, ct);
19
20     public async Task SaveChangesAsync(CancellationToken ct =
default) =>
21         await dbContext.SaveChangesAsync(ct);
22 }

```

Listing 4.10: Separación de contrato e implementación mediante el patrón Repository.

Observer (Múltiples Contextos)

El patrón *Observer* define una dependencia de uno-a-muchos, permitiendo que múltiples suscriptores reaccionen a cambios de estado sin que el emisor deba conocerlos. Dada su versatilidad, este patrón vertebra tres componentes críticos del sistema:

1. Eventos de Dominio: Los agregados acumulan eventos de dominio internamente durante su ciclo de vida. Al invocar `SaveChangesAsync`, el contexto de base de datos extrae estos eventos, los despacha sincrónicamente mediante MediatR y, en la misma transacción, los serializa en la tabla del *Outbox*.

```

1 public abstract class Entity
2 {
3     private readonly List<IDomainEvent> _domainEvents = [];
4     public IReadOnlyCollection<IDomainEvent> DomainEvents =>
_domainEvents.AsReadOnly();
5     public void Raise(IDomainEvent domainEvent) =>
_domainEvents.Add(domainEvent);
6 }
7
8 // Intercepcion del DbContext para despachar observadores
9 public override async Task<int> SaveChangesAsync(
CancellationToken ct = default)
10 {
11     List<IDomainEvent> domainEvents = ChangeTracker.Entries<
Entity>()
12         .SelectMany(e => e.Entity.DomainEvents).ToList();
13
14     await domainEventsDispatcher.DispatchAsync(domainEvents,
ct);
15     InsertOutboxMessages(domainEvents);
16
17     return await base.SaveChangesAsync(ct);
18 }

```

Listing 4.11: Implementación del patrón Observer para Eventos de Dominio.

2. Eventos de Integración (Outbox/Inbox): Constituyen la aplicación *cross-módulo* del *Observer*. Un módulo publica un evento que otros módulos consumen de forma

asíncrona. La combinación de los patrones *Outbox* e *Inbox* asegura que la publicación es atómica respecto a la escritura en base de datos y que la recepción es idempotente.

```

1 public abstract class IntegrationEventHandler<
  TIntegrationEvent, TContext>(...)
2   : INotificationHandler<TIntegrationEvent>
3 {
4   public async Task Handle(TIntegrationEvent notification,
  Cancellation token ct)
5   {
6       Guid messageId = integrationEventContextAccessor.
  CurrentMessageId
7       ?? GetFallbackMessageId(notification);
8
9       // Verificación de idempotencia: actúa como un
  observer seguro
10      bool alreadyProcessed = await dbContext.Set<
  InboxMessage>()
11      .AsNoTracking().AnyAsync(m => m.Id == messageId,
  ct);
12
13      if (alreadyProcessed) return;
14
15      await HandleIntegrationEvent(notification, ct);
16
17      dbContext.Set<InboxMessage>().Add(InboxMessage.Create
  (messageId, ...));
18      await dbContext.SaveChangesAsync(ct);
19  }
20 }

```

Listing 4.12: Consumo asíncrono e idempotente de Eventos de Integración (Inbox).

3. Gestión de Estado en Frontend (React Query): En el cliente, React Query implementa el patrón *Observer* para sincronizar el estado del servidor. Los componentes se suscriben a una *Query*, cuando los datos se invalidan o actualizan en la caché compartida, todos los observadores se re-renderizan automáticamente. Se ha diseñado un sistema de *QueryKeys* jerárquicas para permitir invalidaciones de caché granulares.

```

1 // Estructura jerárquica de claves para invalidación granular
2 export const QueryKeys = {
3   workItem: {
4     management: {
5       all: ['workitem', 'management'] as const,
6       list: (projectId: string) => ['workitem', 'management',
  'list', projectId] as const,
7     },
8   },
9 } as const;
10
11 // Hook observador del estado
12 export const useProjectWorkItemsQuery = (projectId: string |
  null) => {

```

```

13  const accessToken = useAuthStore((state) => state.
    accessToken);
14  return useQuery({
15    queryKey: QueryKeys.workItem.management.list(projectId ??
      ''),
16    queryFn: async () => { /* Logica de paginacion y llamada
      a la API */ },
17    enabled: Boolean(accessToken && projectId),
18  });
19  };

```

Listing 4.13: Suscripción reactiva al estado del servidor mediante React Query.

Chain of Responsibility (Canalización de Autorizaciones)

El patrón *Chain of Responsibility* permite que una petición pase por una cadena de manejadores secuenciales, donde cualquiera de ellos puede detener el flujo si no se cumplen ciertas condiciones. En el sistema de autorización, `OrganizationPermissionBehavior` ejecuta una cadena de validaciones lógicas: verifica la existencia de la organización, la disponibilidad del permiso, la membresía del usuario y, finalmente, su nivel de acceso.

```

1  public sealed class OrganizationPermissionBehavior<TRequest,
    TResponse>(...)
2      : IPipelineBehavior<TRequest, TResponse>
3  {
4      public async Task<TResponse> Handle(TRequest request,
5          RequestHandlerDelegate<TResponse> next,
6          CancellationToken ct)
7      {
8          // [...] Extraccion de la configuracion de permisos
          // requeridos
9
10         Organization? org = await organizationRepository
            .GetByIdAsync(permissionRequest.OrganizationId,
11             ct);
12         if (org is null) return CreateFailureResponse(
            OrganizationNotFound); // Corta la cadena
13
14         OrganizationInvitation? invitation = org.Investigations
            .FirstOrDefault(inv => inv.UserId == userContext.
            UserId);
15         if (invitation is null) return CreateFailureResponse(
            OrganizationMembershipRequired);
16
17         // [...] Evaluacion del nivel del rol
18         if (rolePermission?.Level == PermissionLevel.Full)
19             return await next(ct); // Autorizado: pasa el
            control al siguiente eslabon
20
21         return CreateFailureResponse(
            OrganizationPermissionDenied);
22     }

```

23 }

Listing 4.14: Validación en cascada mediante Chain of Responsibility.

Command (Mutaciones en Frontend)

El patrón *Command* encapsula una solicitud y sus efectos secundarios como un objeto independiente. En el *frontend*, las mutaciones de React Query actúan como comandos puros: agrupan la llamada a la API, el manejo de errores, la invalidación de la caché local y la retroalimentación visual al usuario en una unidad funcional encapsulada.

```

1 export const useCreateWorkItemMutation = (projectId: string)
  => {
2   const handleApiError = useApiErrorHandler();
3
4   return useMutation({
5     mutationKey: [...QueryKeys.workItem.management.all, '
      create', projectId],
6     mutationFn: (payload: CreateWorkItemRequest) =>
7       ApiGateway.workItem.management.createWorkItem(projectId
      , payload),
8     onSuccess: async () => {
9       // Efectos secundarios unificados dentro del Command
10      await queryClient.invalidateQueries({
11        queryKey: QueryKeys.workItem.management.list(
12        projectId)
13      });
14      toast.success(i18n.t('create.feedback.created', { ns: '
      workitems' }));
15    },
16    onError: (error) => handleApiError(error),
17  });

```

Listing 4.15: El patrón Command encapsulando operaciones de escritura en React Query.

Strategy (Inicialización Dinámica de Datos)

El patrón *Strategy* define una familia de algoritmos, los encapsula y los hace intercambiables en tiempo de ejecución sin alterar el cliente que los utiliza. La inicialización de la base de datos (*Seeding*) utiliza este patrón inyectando diferentes estrategias (DevelopmentUserSeeder frente a ProductionUserSeeder) en función del entorno de despliegue, evitando lógicas condicionales distribuidas.

```

1 public interface IUserSeederStrategy
2 {
3   Task SeedAsync(UserDbContext dbContext, CancellationToken
      ct = default);
4 }
5

```

```

6 public sealed class DevelopmentUserSeeder :
  IUserSeederStrategy
7 {
8     public async Task SeedAsync(UserDbContext dbContext,
  CancellationTokens ct = default)
9     {
10         if (await dbContext.Users.AnyAsync(ct)) return;
11         User[] users = [
12             User.Create("Admin", "Developer", "admin@dev.com"
13         ),
14             User.Create("John", "Doe", "john@dev.com")
15         ];
16         await dbContext.Users.AddRangeAsync(users, ct);
17         await dbContext.SaveChangesAsync(ct);
18     }
19 }
20 // Inyección de dependencias resolviendo la estrategia en el
  arranque
21 services.AddScoped<IUserSeederStrategy>()
22     .Add(isDevelopment
23         ? _ => new DevelopmentUserSeeder()
24         : _ => new ProductionUserSeeder());

```

Listing 4.16: Implementación de Strategy para la inyección de datos iniciales.

Template Method (Ganchos de Consulta Uniformes)

El patrón *Template Method* define el esqueleto de un algoritmo, delegando la implementación de ciertos pasos a subclases o instancias específicas. En los *hooks* de consulta del cliente, se define una plantilla inmutable (comprobación de identidad → asignación de clave → llamada de red → condición de activación) aplicable a todos los recursos del dominio, especializando únicamente las propiedades locales.

```

1 // Plantilla estructural que todos los query hooks del
  dominio replican
2 export const use[Domain]Query = (id: string | null) => {
3     const accessToken = useAuthStore((state) => state.
      accessToken);
4
5     return useQuery({
6         queryKey: QueryKeys.[domain].list(id ?? ''), //
7         queryFn: () => ApiGateway.[domain].get(id!), //
8         enabled: Boolean(accessToken && id), //
9         // Condición común
10    });
11 };

```

Listing 4.17: Estructura Template Method aplicada a los custom hooks de React.

4.5 Diseño de la interfaz de usuario

El diseño de la UI de Leaftask se concibió con el objetivo principal de ofrecer una experiencia fluida y reducir la carga cognitiva del usuario al enfrentarse a un sistema con alta densidad de información (gestión de proyectos jerárquicos e interacciones con inteligencia artificial) [22]. Para lograr esto, se establecieron patrones de interacción consistentes que priman la claridad y el pragmatismo sobre la ornamentación visual.

4.5.1 Decisiones de diseño e implementación

El sistema visual de la aplicación se construyó utilizando la biblioteca de componentes *shadcn/ui*, combinada con el motor de estilos Tailwind CSS. Esta decisión técnica, similar al uso de utilidades atómicas en el desarrollo móvil, ha permitido estandarizar de forma centralizada los espaciados, la paleta cromática y las tipografías, encapsulando los estilos en un sistema unificado y limpio sin generar una arquitectura de hojas de estilo inmanejable.

La navegación espacial se resolvió adoptando un patrón de Panel Lateral Jerárquico. Este enfoque garantiza la heurística de "Visibilidad del estado del sistema" [3], separando claramente los accesos globales (perfil, notificaciones, chats) en la parte inferior, de los controles contextuales del proyecto activo en la parte superior.

4.5.2 Visualización de datos y complejidad

El mayor reto de la interfaz consistía en representar jerarquías de tareas complejas. Para ello, el sistema abandona las tablas tradicionales en favor de representaciones visuales que permiten un escaneo cognitivo rápido:

- **Mapeo jerárquico (*Tree View*):** La vista principal del proyecto (Figura 4.8) se abstrae en un grafo interactivo bidimensional. Los elementos de trabajo (*Work Items*) se representan como nodos con dependencias visuales explícitas (líneas conectoras). El color y el relleno de los nodos actúan como barras de progreso y estado radiales, siguiendo lo definido en el capítulo de análisis.
- **Gestión del contexto:** Al interactuar con un nodo, un panel lateral derecho se despliega con el detalle completo de la tarea, evitando la pérdida de contexto que supondría una redirección a una nueva página completa.

4.5.3 Flujo de comunicación con el usuario

El flujo de comunicación define cómo el sistema interactúa, responde y guía al usuario. Para garantizar una experiencia predecible, el diseño se fundamenta en tres pilares:

- **Gestión de la espera asíncrona:** En operaciones que requieren llamadas complejas al *backend* o procesamiento de la Inteligencia Artificial (Figura 4.10), el sistema enmascara la latencia mediante comunicación visual. Los agentes integrados en el chat informan proactivamente de las acciones que van a ejecutar ("Voy a actualizar el estado del *work item*..."), transformando el tiempo de procesamiento en una experiencia conversacional fluida.

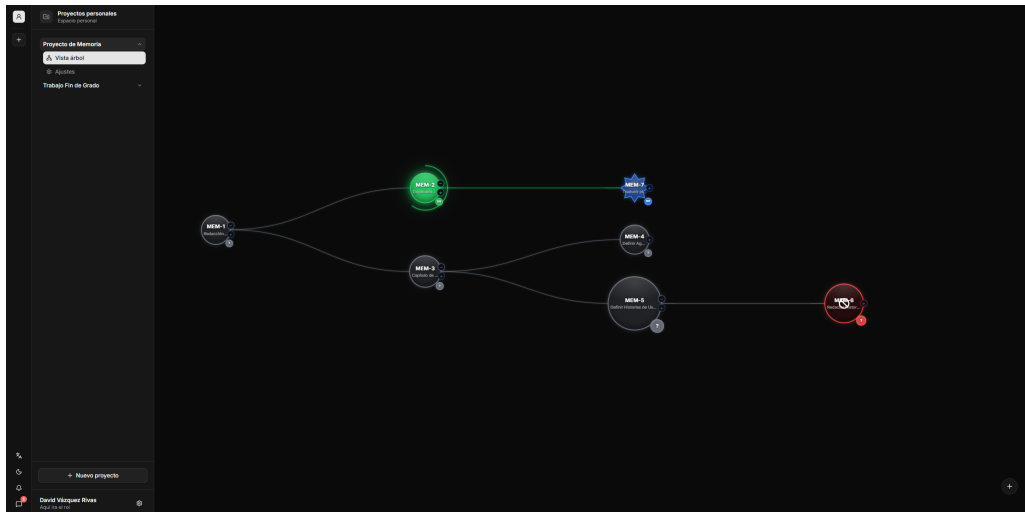


Figura 4.8: Vista en árbol del proyecto, mostrando la jerarquía y dependencias de las tareas.

- **Confirmaciones y zonas de peligro:** La plataforma informa proactivamente sobre el resultado de las operaciones mediante notificaciones efímeras (*toasts*) en las esquinas de la pantalla. Adicionalmente, las acciones destructivas se aíslan visualmente en Zonas de Peligro (*Danger Zones*) y requieren confirmación explícita.
- **Prevención de errores en autorizaciones:** El sistema materializa el modelo complejo de roles (Figura 4.9) en una interfaz tabular con semántica de colores clara (verde para control completo, amarillo para acciones supervisadas), minimizando el riesgo de que los administradores apliquen configuraciones de seguridad erróneas.

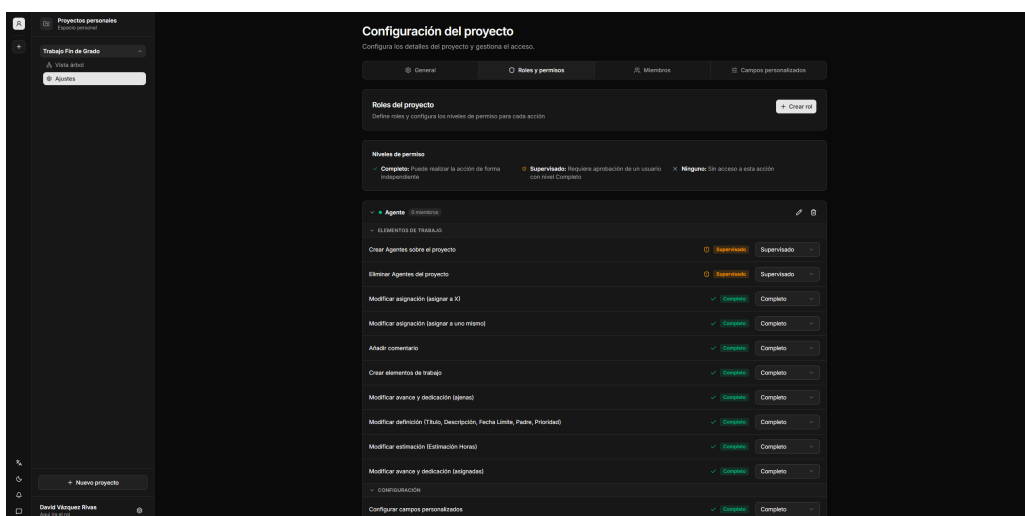


Figura 4.9: Interfaz de configuración de roles y permisos del proyecto.

4. DISEÑO

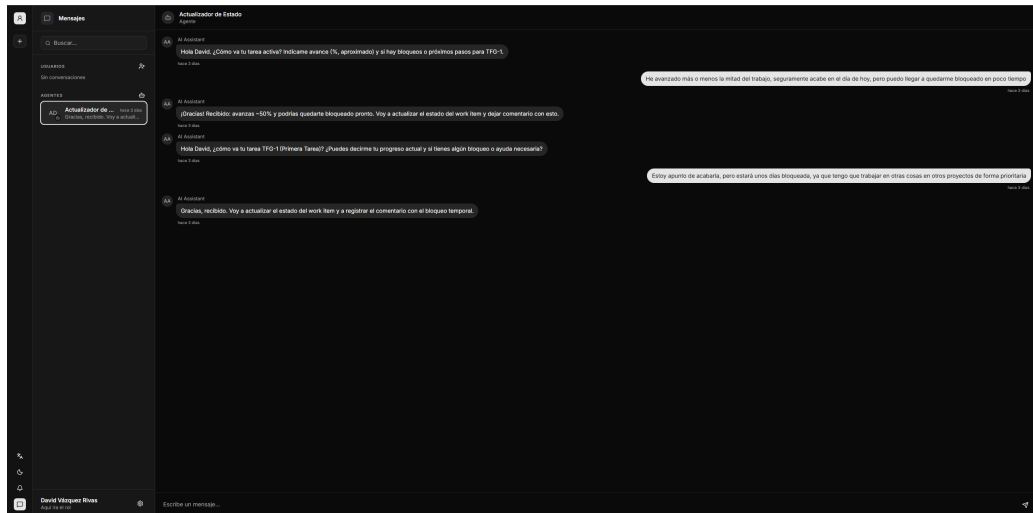


Figura 4.10: Interfaz de mensajería mostrando una conversación natural con un Agente de IA.

PLANIFICACIÓN

5.1 Gestión del ciclo de vida

Para el desarrollo de LeafTask, se ha descartado el uso de metodologías tradicionales o predictivas en cascada (*Waterfall*). En su lugar, se ha adoptado un modelo de ciclo de vida iterativo e incremental basado en los principios ágiles [23], adaptado a la realidad y a los tiempos de un Trabajo de Fin de Grado desarrollado de forma individual.

Este enfoque metodológico ha permitido mantener una gran flexibilidad ante la resolución de retos técnicos complejos (como la arquitectura orientada a eventos o la integración de IA), garantizando que las decisiones de diseño se pudieran auditar y pivotar de forma continua junto al tutor.

5.1.1 Organización del proyecto y Sprints

La planificación temporal del proyecto se ha estructurado mediante *sprints*. Como norma general, se ha establecido una duración fija de **dos semanas por *sprint***, lo que proporcionó una (*cadencia*) predecible y constante. Esta duración estándar solo sufrió alteraciones justificadas en momentos puntuales del calendario debido a periodos vacacionales o festividades.

La evolución operativa a lo largo de estos *sprints* se dividió conceptualmente en tres grandes etapas:

1. **Fase de Inicio e Investigación (*Sprints* 1 a 4 – Semanas 1 a 8):** Las primeras cuatro semanas (*Sprints* 1 y 2) se dedicaron al análisis exhaustivo de requisitos y al estudio de viabilidad tecnológica (evaluación de LLM y protocolos Model Context Protocol (MCP)). Las dos semanas siguientes (*Sprint* 3) se centraron en el diseño arquitectónico C4 y el modelado del dominio. Finalmente, el *Sprint* 4 abordó simultáneamente la configuración de la infraestructura contenerizada (base de proyecto frontend y backend), la definición de la API REST (contrato)

5. PLANIFICACIÓN

para finalizar con el diseño y el desarrollo completo del primer *Bounded Context*: el módulo de **Identidad y Accesos**, requisito bloqueante para el resto del sistema.

2. **Fase de Desarrollo (*Sprints* 5 a 9 – Semanas 9 a 20):** Representa el núcleo de codificación de la plataforma. Para garantizar la entrega continua, la implementación de los módulos funcionales se secuenció priorizando la resolución de dependencias arquitectónicas:

- ***Sprint* 5:** Desarrollo del módulo de **Organizaciones**, estableciendo la estructura base.
- ***Sprint* 6:** Desarrollo del módulo de **Proyectos**.
- ***Sprint* 7:** Desarrollo del módulo central de **Elementos de Trabajo** (*Work Items*), materializando la lógica del Árbol de Trabajo.
- ***Sprint* 8:** Construcción del módulo de **Chats**. Durante esta iteración se tomó la decisión de adelantar el análisis y diseño de la base del módulo de **Agentes de IA**, con el objetivo de finalizar las interfaces de comunicación del chat de forma temprana y evitar cuellos de botella.
- ***Sprint* 9:** Cierre definitivo del motor de **Agentes de IA**. En esta misma iteración se completó el módulo de **Notificaciones**, absorbiendo la carga de trabajo de forma paralela para acelerar la finalización de los agentes.

3. **Fase de Cierre y Documentación (*Sprints* 9 y 10 – Semanas 19 a 22):** Para optimizar el calendario y mitigar riesgos, la redacción de esta memoria comenzó solapándose con el último sprint de desarrollo, el *Sprint* 9. El último bloque funcional (*Sprint* 10) se reservó en exclusiva para el cierre de la documentación y la estabilización final del sistema.

5.1.2 Diagrama de Gantt

Para visualizar formalmente la distribución de los *Sprints*, la secuenciación de los módulos y los solapamientos temporales descritos, la Figura 5.1 representa el cronograma global del proyecto.

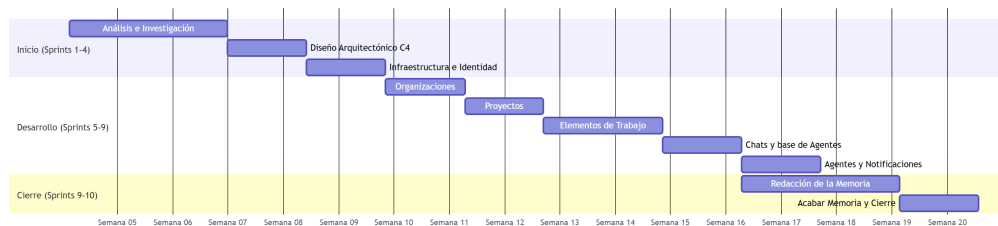


Figura 5.1: Diagrama de Gantt del proyecto distribuido por semanas lógicas de trabajo.

5.1.3 Flujo de trabajo y seguimiento de tareas

Para el seguimiento diario y la gestión de las tareas dentro de cada *sprint*, se adoptó un enfoque visual basado en tableros Kanban. Este sistema permitió mantener un control absoluto sobre el estado funcional del proyecto y el avance de la carga de trabajo.

Cada tarea o elemento de trabajo transitaba por un ciclo de vida definido por cinco estados estrictos, garantizando un flujo ordenado desde su concepción hasta su integración final:

- **Por hacer (*To Do*):** Tareas y requisitos técnicos que han sido analizados, definidos y asignados a la iteración actual, listos para comenzar su desarrollo.
- **En curso (*In Progress*):** El elemento de trabajo se encuentra en fase de codificación o desarrollo activo.
- **Por revisar (*To Review*):** El desarrollo técnico ha concluido a nivel local. El código ha sido subido al repositorio y se encuentra a la espera de ser sometido a validaciones (como la ejecución de pruebas automáticas en el flujo de integración continua).
- **En revisión (*In Review*):** Fase de auditoría del código. En este estado se revisan activamente los cambios implementados para asegurar que cumplen con los estándares arquitectónicos del proyecto y los requisitos del dominio antes de ser integrados en la rama principal.
- **Finalizado (*Done*):** La tarea ha superado todas las validaciones de calidad, ha sido integrada (*merged*) en el sistema y se considera completamente terminada.

5.2 Aseguramiento de la calidad y pruebas

Se ha dado prioridad a la calidad en el desarrollo del sistema, pero en lugar de forzar una metodología de Desarrollo Guiado por Pruebas (TDD) estricta en todo el proyecto, se adoptó una estrategia de calidad pragmática y focalizada en el núcleo del sistema, alineada con los principios de la *Clean Architecture*.

Se estableció como requisito técnico innegociable alcanzar un **100% de cobertura de código** mediante pruebas unitarias en las dos capas más críticas del *backend*: la capa de **Dominio** y la capa de **Aplicación**. Esta exigencia garantizó que todas las reglas de negocio, validaciones de estado de los agregados y la orquestación de los casos de uso funcionaran de forma impecable y estuvieran blindadas frente a regresiones durante la evolución del sistema. Las capas externas (como la de infraestructura o controladores de API) se cubrieron mediante pruebas de integración selectivas, optimizando así el tiempo de desarrollo sin comprometer la solidez de la lógica de negocio central.

5.3 Definición del *Backlog*

El *Backlog* ha actuado como la única fuente de la verdad para centralizar todos los requisitos, mejoras funcionales y tareas técnicas del proyecto. Lejos de ser un documento estático, se ha gestionado como un listado vivo y dinámico, permitiendo incorporar

los descubrimientos técnicos y adaptando el alcance a medida que la arquitectura del sistema evolucionaba.

5.3.1 Estructuración y jerarquía de elementos

Para evitar la ambigüedad y mantener el alineamiento entre la visión global del producto y el código fuente, los elementos del *backlog* se han estructurado en tres niveles de granularidad:

- **Épicas (*Epics*):** Representan grandes bloques de funcionalidad o áreas de negocio que, por su tamaño, no pueden ser completadas en un solo *sprint*. En Leaftask, las Épicas se mapearon directamente de forma unívoca con los *Bounded Contexts* identificados en la fase de análisis.
- **Historias de Usuario:** Descomponen las Épicas en unidades de valor funcional e independiente desde la perspectiva del usuario o del sistema. Son las identificadas anteriormente en el análisis y cada una cuenta con sus propios Criterios de Aceptación (*Definition of Done*), los cuales sirvieron como base para el diseño de las pruebas automatizadas.
- **Tareas Técnicas (*Tasks*):** El nivel más granular, utilizado durante la planificación del *sprint*. Consisten en pasos de ingeniería específicos necesarios para completar una historia (por ejemplo, "Crear migración de base de datos para la tabla de proyectos").

5.3.2 Criterios de priorización

Dada la complejidad estructural de un monolito modular, la priorización de los elementos del *backlog* no se basó exclusivamente en el valor de negocio final, sino en la **mitigación temprana de riesgos arquitectónicos**.

5.3.3 Estimación de esfuerzo

La estimación del esfuerzo necesario para cada tarea se realizó en horas de dedicación efectiva. Al tratarse de un desarrollo individual, el uso de métricas relativas (como los Puntos de Historia) perdía su propósito de consenso en equipo. En su lugar, la estimación temporal en horas permitió auditar de forma precisa la desviación entre el tiempo planificado en las sesiones de análisis y el tiempo real de codificación invertido al cerrar cada *sprint*. Esta métrica de desviación sirvió como mecanismo de *feedback* continuo para ajustar la carga de trabajo en las iteraciones posteriores de forma cada vez más realista.

5.4 *Definition of Ready* y *Definition of Done*

Para garantizar la calidad del proyecto y evitar cuellos de botella el desarrollo, se han establecido dos políticas de calidad fundamentales derivadas del marco de trabajo Scrum: la *Definition of Ready* y la *Definition of Done*. Estas políticas actúan como contratos que estandarizan el flujo de trabajo de cualquier elemento del *Backlog*.

5.4.1 *Definition of Ready*

La *Definition of Ready* establece los requisitos previos e innegociables que debe cumplir un elemento de trabajo (*Work Item*) o Historia de Usuario antes de poder ser asignado a un *sprint* y comenzar su codificación. Su objetivo es asegurar que el trabajo está lo suficientemente detallado como para ser implementado sin ambigüedades.

Para que una tarea sea considerada como *Ready*, debe cumplir los siguientes criterios:

- **Descripción clara:** La tarea posee una descripción comprensible que detalla el objetivo funcional o técnico a resolver.
- **Criterios de aceptación:** Se han definido explícitamente las condiciones que el sistema debe cumplir para que la funcionalidad se considere exitosa.
- **Estimación asignada:** La tarea ha sido analizada y cuenta con una estimación de esfuerzo en horas.

5.4.2 *Definition of Done*

La *Definition of Done* dicta el estándar de calidad unificado que debe alcanzar cualquier incremento de código para ser considerado como finalizado y listo para ser integrado en la rama principal (*main*) del repositorio. Esta política es vital para mantener la deuda técnica bajo control en una arquitectura compleja.

Un elemento de trabajo solo transiciona al estado finalizado (*Done*) cuando satisfice íntegramente la siguiente lista de verificación:

- **Criterios de aceptación superados:** El código implementado resuelve satisfactoriamente todos los puntos definidos en la DoR de la tarea.
- **Cobertura de pruebas (*Testing*):** Para las tareas de *backend*, se han implementado las pruebas unitarias correspondientes y se mantiene la exigencia del 100 % de cobertura de código en las capas de Dominio y Aplicación. Además, todos los *tests* previos del sistema se ejecutan con éxito, garantizando la ausencia de regresiones.
- **Análisis de código estático:** El código cumple con las normativas de estilo, nomenclatura y principios de la *Clean Architecture* establecidos para el proyecto, sin presentar advertencias (*warnings*) críticas por parte del compilador o del *linter*.

5. PLANIFICACIÓN

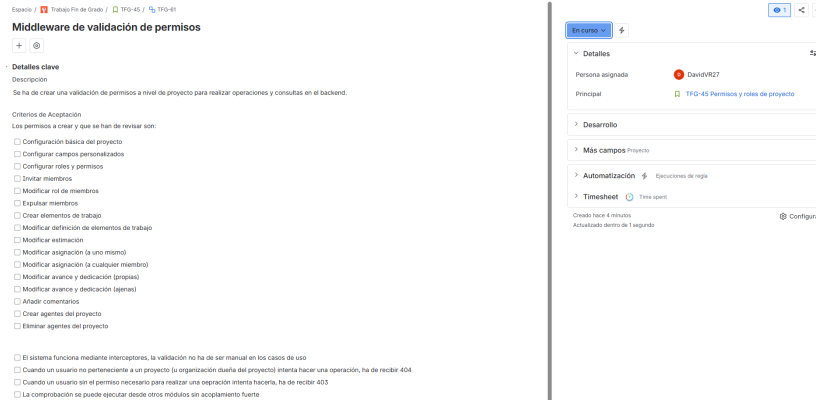


Figura 5.2: Ejemplo de un elemento de trabajo (*Work Item*) en curso.

5.5 Camino crítico y análisis de dependencias

Para garantizar el cumplimiento de los plazos y gestionar adecuadamente las interdependencias entre los diferentes módulos del sistema, se ha modelado la red de tareas del proyecto mediante un diagrama PERT (*Program Evaluation and Review Technique*) y se ha evaluado utilizando el Critical Path Method (CPM) [24].

El modelo de dependencias lógicas, ilustrado en la Figura 5.3, revela que la arquitectura impone restricciones estrictas de precedencia. Por ejemplo, es inviable implementar la lógica de negocio funcional sin haber estabilizado previamente la infraestructura base (T6) y el módulo de Identidad (T7), dado que todos los agregados del sistema requieren asociarse a un identificador de usuario válido.

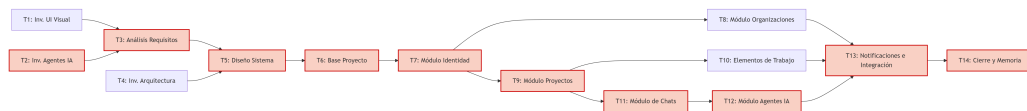


Figura 5.3: Diagrama PERT del proyecto. El camino crítico se resalta en color rojo.

El camino crítico representa la secuencia más larga de tareas dependientes, dictaminando la duración mínima total del proyecto. Cualquier desviación temporal en estas actividades impactaría directamente en la fecha de entrega nula. Tras el análisis del grafo de dependencias, el camino crítico identificado es el siguiente:

$$T2 \rightarrow T3 \rightarrow T5 \rightarrow T6 \rightarrow T7 \rightarrow T9 \rightarrow T11 \rightarrow T12 \rightarrow T13 \rightarrow T14$$

La justificación técnica de esta ruta crítica radica en dos cuellos de botella fundamentales:

1. **Investigación de IA frente a UI (T2 vs T1):** En la fase de inicio, la complejidad inherente al estudio de los LLM y el protocolo MCP (T2) supuso un mayor esfuerzo que la definición de interfaces visuales (T1), convirtiendo a la primera en el factor limitante para iniciar el análisis global (T3).
2. **Cadena de dependencia funcional (T9 → T11 → T12):** Una vez finalizado el módulo de Identidad (T7), el grafo se ramifica. Sin embargo, la rama de mayor

profundidad es la que involucra a los agentes autónomos. Para que un agente (T12) pueda operar en el sistema, requiere que el módulo de Proyectos (T9) exista para tener un contexto, y que el módulo de Chats (T11) esté finalizado para disponer de una interfaz de comunicación (*endpoints* y persistencia de mensajes).

Por el contrario, el diagrama revela que actividades como el módulo de Organizaciones (T8) o el propio Árbol de Elementos de Trabajo (T10) no forman parte del camino crítico. Aunque son componentes centrales de la aplicación, su desarrollo discurre en paralelo a la rama de los agentes de IA.

Es importante destacar que, aunque con este análisis se ha identificado un camino crítico y módulos o tareas paralelizables, al tratarse de un proyecto individual, la ejecución de las tareas no puede realizarse en paralelo. Por lo tanto, la planificación temporal se ha basado en la secuenciación de los módulos y no en la paralelización de tareas.

5.6 Políticas de versionado y gestión del código fuente

El control de versiones es un pilar fundamental en la ingeniería de *software* moderna, garantizando la trazabilidad de los cambios, la recuperación ante errores y la organización del trabajo. Para la gestión del código fuente de Leaftask, se ha utilizado Git como sistema de control de versiones distribuido, alojando el repositorio central de forma remota.

5.6.1 Modelo teórico y políticas de protección

En su concepción teórica, y pensando en la escalabilidad futura hacia un equipo de desarrollo completo, la gestión de ramas se planificó tomando como referencia el estándar *Git Flow* [?]. Este modelo define una estructura estricta basada en dos ramas principales de vida infinita y múltiples ramas efímeras:

- **main**: Rama principal que contiene exclusivamente código estable, probado y listo para producción. Cada integración en esta rama representa una versión entregable (*release*) al final de un *sprint*.
- **develop**: Rama de integración continua. Actúa como el tronco principal de trabajo donde se unifican las nuevas funcionalidades antes de pasar a producción.
- **Ramas de característica (feature/*)**: Ramas efímeras creadas a partir de **develop** para desarrollar tareas individuales del *backlog*.

Bajo este paradigma, se definieron políticas de protección de ramas para evitar la introducción de código inestable. Estas políticas prohíben la subida directa de código a las ramas **main** y **develop**, obligando a que cualquier integración se realice mediante una *Pull Request*. En un entorno corporativo, esta solicitud requeriría la revisión y aprobación cruzada de al menos otro desarrollador (*Peer Review*) y la ejecución exitosa de las pruebas automáticas.

5.6.2 Adaptación pragmática: *Trunk-Based Development*

A pesar de la solidez del modelo teórico planteado, la aplicación estricta de *Git Flow* y la auto-revisión de *Pull Requests* en un proyecto desarrollado por un único individuo introduce una sobrecarga burocrática innecesaria, penalizando la agilidad sin aportar un beneficio real en la detección de errores.

Por este motivo, durante la fase de ejecución, la política de versionado se adaptó hacia un enfoque más pragmático inspirado en *Trunk-Based Development*. Las reglas operativas aplicadas fueron las siguientes:

- **Integración directa en desarrollo:** La rama `develop` se utilizó como el tronco único de trabajo diario (*trunk*). Los incrementos de código, correspondientes a las tareas del *sprint*, se integraron directamente sobre esta rama mediante *commits* atómicos y descriptivos, omitiendo la creación constante de ramas `feature/*`.
- **Validación local continua:** Al eliminar la barrera de las *Pull Requests* intermedias, la responsabilidad de mantener la estabilidad de `develop` recayó en la ejecución local y continua de la batería de pruebas unitarias antes de cada *commit*.
- **Entregables por iteración:** Al finalizar cada iteración, y tras la revisión del *sprint* y comprobar la integridad del sistema, la rama `develop` se fusionó (*merge*) con la rama `main`. Esta fusión generó incrementos estables sobre la rama `main`, manteniendo así la filosofía de entrega continua.



ANEXOS

Texto placeholder de anexos. Aquí se añadirá material complementario cuando la memoria esté más avanzada.

BIBLIOGRAFÍA

- [1] D. Hardt, “The oauth 2.0 authorization framework,” RFC Editor, RFC 6749, 2012.
- [2] R. C. Martin, *Clean Architecture: A Craftsman’s Guide to Software Structure and Design*. Prentice Hall, 2017.
- [3] J. Nielsen and R. Molich, “Heuristic evaluation of user interfaces,” in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. ACM, 1990, pp. 249–256.
- [4] W3C World Wide Web Consortium, “Web content accessibility guidelines (wcag) 2.1,” <https://www.w3.org/TR/WCAG21/>, 2018.
- [5] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley Professional, 2004.
- [6] M. Fowler, “Bounded context,” <https://martinfowler.com/bliki/BoundedContext.html>, 2014, accedido: 11 de junio de 2026.
- [7] P. P.-S. Chen, “The entity-relationship model: Toward a unified view of data,” *ACM Transactions on Database Systems (TODS)*, vol. 1, no. 1, pp. 9–36, 1976.
- [8] M. Fowler, “Cqrs (command query responsibility segregation),” <https://martinfowler.com/bliki/CQRS.html>, 2011, accedido: 9 de junio de 2026.
- [9] S. Brown, “The c4 model for visualising software architecture,” <https://c4model.com/>, 2026, accedido: 10 de junio de 2026.
- [10] S. Newman, *Monolith to Microservices: Evolutionary Patterns to Transform Your Monolith*. O’Reilly Media, 2019.
- [11] G. Hohpe and B. Woolf, *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley, 2004.
- [12] S. Gilbert and N. Lynch, “Brewer’s conjecture and the feasibility of consistent, available, partition-tolerant web services,” *ACM SIGACT News*, vol. 33, no. 2, pp. 51–59, 2002.
- [13] A. Cockburn, “Hexagonal architecture,” <https://alistair.cockburn.us/hexagonal-architecture/>, 2005, accedido: 11 de junio de 2026.
- [14] R. C. Martin, “The dependency inversion principle,” *C++ Report*, vol. 8, no. 5, pp. 61–66, 1996.

- [15] J. Bogard, “Vertical slice architecture,” <https://jimmybogard.com/vertical-slice-architecture/>, 2018, accedido: 11 de junio de 2026.
- [16] B. Frost, *Atomic Design*. Brad Frost Web, 2016.
- [17] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional, 1994.
- [18] T. Linsley and TanStack Contributors, “Tanstack query: Powerful asynchronous state management,” <https://tanstack.com/query/latest>, 2026, accedido: 11 de junio de 2026.
- [19] C. Richardson, *Microservices Patterns: With examples in Java*. Manning Publications, 2018.
- [20] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “React: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations (ICLR)*, 2023.
- [21] M. Jones, J. Bradley, and N. Sakimura, “Json web token (jwt),” RFC Editor, RFC 7519, 2015.
- [22] J. Sweller, “Cognitive load during problem solving: Effects on learning,” *Cognitive Science*, vol. 12, no. 2, pp. 257–285, 1988.
- [23] K. Beck, M. Beedle, A. van Bennekum, A. Cockburn, W. Cunningham, M. Fowler, J. Grenning, J. Highsmith, A. Hunt, R. Jeffries, J. Kern, B. M. surface, R. C. Martin, S. Mellor, K. Schwaber, J. Sutherland, and D. Thomas, “Manifesto for agile software development,” <https://agilemanifesto.org/>, 2001, accedido: 11 de junio de 2026.
- [24] J. E. Kelley and M. R. Walker, “Critical-path planning and scheduling,” in *Papers Presented at the ACM AIEE-IRE '59 Eastern Joint Computer Conference*. ACM, 1959, pp. 160–173.